Doom Emacs Configuration

Emacs configuration for work and life!

Dang Quang Vu*

September 20, 2022

Contents

1 Initial repository **5**

2 Doom Configuration Files **5**
2.1 Intro 5
2.2 My Private Doom Modules 6
2.3 Config files 6
2.4 Pseudo early-init 7
2.4.1 Useful functions 8
2.4.2 Fixes 11
2.4.3 Check for external tools 11
2.5 Doom modules (`init.el`) 12
2.6 Additional packages (`packages.el`) 17

3 General Emacs Setting **17**
3.1 System/User/Default information 17
3.2 Common Variables 19
3.3 Common Function 19
3.3.1 Show list of buffer when splitting 19
3.3.2 mine 19
3.3.3 Scroll page 22
3.4 Undo 22
3.4.1 Undo-fu 22
3.4.2 Visual undo (`vundo`) 22

*vugomars@gmail.com

3.5 Messages buffer . . . 23
3.6 Secrets . . . 24

4 Emacs daemon **24**
4.1 Initialization . . . 24
4.2 Tweaks . . . 25
4.2.1 Save recent files . . . 25

5 User Interface **26**
5.1 Font . . . 26
5.2 Theme . . . 26
5.2.1 Doom . . . 26
5.2.2 Modus themes . . . 26
5.3 Mode line . . . 28
5.4 Set transparency & Full Screen . . . 29
5.5 Dashboard . . . 29
5.6 SVG tag and `svg-lib` . . . 30
5.7 Focus . . . 30
5.8 Scrolling . . . 31
5.9 All the icons . . . 31
5.10 Tabs . . . 31
5.11 Zen (writeroom) mode . . . 32
5.12 Highlight indent guides . . . 32

6 Keybindings **33**
6.1 Disable keybindings . . . 33
6.2 F1~12(kbd) . . . 33
6.3 Common . . . 33
6.4 SPC leader . . . 35
6.5 org-mode . . . 37
6.6 deadgrep-mode . . . 39

7 Package Configuration **39**
7.1 Editing . . . 39
7.1.1 File templates . . . 39
7.1.2 Scratch buffer . . . 40
7.1.3 Very large files . . . 40
7.1.4 Evil . . . 40
7.1.5 Aggressive indent . . . 40
7.1.6 YASnippet . . . 40

7.2 Symbols . . . 41
7.2.1 Emojify . . . 41
7.2.2 Ligatures . . . 42
7.3 Features . . . 42
7.3.1 Workspaces . . . 42
7.3.2 Info colors . . . 43
7.3.3 The Silver Searcher . . . 43
7.3.4 Page break lines . . . 44
7.3.5 PDF tools . . . 44
7.4 Natural languages . . . 46
7.4.1 Spell-Fu . . . 46
7.4.2 Proselint . . . 46
7.4.3 Grammarly . . . 47
7.4.4 Grammalecte . . . 49
7.4.5 LTeX/LanguageTool . . . 50
7.4.6 Go Translate (Google, Bing and DeepL) . . . 51
7.4.7 Offline dictionaries . . . 53

8 Completion & IDE **54**
8.1 UI Package . . . 54
8.1.1 Which key . . . 54
8.1.2 Company . . . 54
8.1.3 Treemacs . . . 56
8.1.4 Projectile . . . 58
8.1.5 Tramp . . . 58
8.1.6 Eros-eval . . . 59
8.1.7 `dir-locals.el` . . . 59
8.1.8 Erefactor . . . 59
8.1.9 Lorem ipsum . . . 60
8.1.10 Coverage test . . . 60
8.2 Languages . . . 60
8.2.1 Language Server Protocol . . . 60
8.2.2 C++ . . . 64
8.2.3 Solidity . . . 66
8.3 Debugging . . . 67
8.3.1 DAP . . . 67
8.4 Development Config . . . 68
8.4.1 multi-iedit . . . 68
8.4.2 engine-mode . . . 68
8.4.3 leetcode . . . 70

8.4.4 bm . . . 70

9 Application **72**
9.1 Maxima . . . 72
9.1.1 Maxima . . . 72
9.1.2 IMaxima . . . 72

10 Programming **73**
10.1 CSV rainbow . . . 73
10.2 Python IDE . . . 74
10.3 Scheme . . . 74
10.4 Git & VC . . . 74
10.4.1 Magit . . . 74
10.4.2 Repo . . . 76
10.4.3 Blamer . . . 76
10.5 ESS . . . 77
10.6 Semgrep . . . 77
10.7 Assembly . . . 77
10.8 Devdocs . . . 78
10.9 PKGBUILD . . . 78
10.10 Flycheck + Projectile . . . 78
10.11 Graphviz . . . 79
10.12 Mermaid . . . 79
10.13 LaTeX . . . 79

11 Office **80**
11.1 Org additional packages . . . 80
11.2 Org mode . . . 81
11.2.1 Intro . . . 81
11.2.2 Behavior . . . 81
11.2.3 Custom links . . . 98
11.2.4 Visuals . . . 99
11.2.5 Bibliography . . . 109
11.2.6 Exporting . . . 111
11.3 Text editing . . . 116
11.3.1 Plain text . . . 116
11.3.2 Academic phrases . . . 117
11.3.3 Yanking multi-lines paragraphs . . . 117

12 Other Config **118**
12.1 Rust format 118
12.2 Enabled some packages 119

1 Initial repository

- This repository (vugomars/doom-config) contains my configuration file for **Zsh**, **Emacs doom**, **Vim**, **Alacritty**.

This is the main configuration file doom-config/config.org, it contains the literal configuration for Doom Emacs, and I use it to generate some other user configuration files.

- Repository này (vugomars/doom-config) Chứa toàn bộ cấu hình của tôi về **Zsh**, **Emacs doom**, **Vim**, **Alacritty**.

File quang trọng nhất của thư mục cấu hình này là doom-config/config.org, Nó chứa cấu hình mọi thứ về Doom Emacs và tôi sử dụng nó để cấu hình cho nhiều file & ứng dụng khác.

2 Doom Configuration Files

2.1 Intro

- I've been using Linux from 2015, **GNU EMACS** was always installed on my from 2018, but i didn't discover the **real** Emacs until 2019. In the beginning, I started Emacs Vanilla, but after a while i found `Doom Emacs`, and since i didn't quit my Emacs Screen!

In the beginning, I Was basically copy chunks of emacs lisp code from the internet, which quickly becomes a mess, specially because i aws using a mixture of vanilla Emacs styles configuration and Doom emacs ones. Now I decided to rewrite a cleaner version of my configuration which will be more Doom friendly.

- Tôi sử dụng Linux từ năm 2015, và **GNU EMACS** luôn là một ứng dụng được cài đặt vào máy tôi kể từ 2018, nhưng tôi thực sự không sử dụng nó quá làm ứng dụng chính. Sau năm 2019, tôi bắt đầu với Emacs Vanilla nhưng sau này tôi sử dụng `Doom emacs`, kể từ đó tôi bắt đầu không thể sử dụng một Text Editor hay IDE nào khác.

- Phiên bản này tôi viết với phiên bản gọn gàng nhất về cấu hình để mọi người có thể tiếp cận nó một cách dễ dàng.

2.2 My Private Doom Modules

I'm moving the big reusable configuration parts to `.doom.d/modules/private`

2.3 Config files

```
;;; config.el -*- coding: utf-8-unix; lexical-binding: t; -*-
```

- Thêm description vào `setup.sh` file, Tệp này sẽ được sử dụng để thiết lập cài đặt hệ thống và cài đặt một số dependencies.

```
#!/bin/bash

# This is an automatically generated setup file, it installes some missing
# dependencies, configure system services, set system settings form better
# desktop integration... etc.
# Dang Quang Vu <vugomars@gmail.com> (c) 2022
```

- Add an initial comment & default enviroment to the `~/.zshrc` file.

```
# -*- mode: sh; -*-

# This file is automatically generated from my Org literate configuration.
# Dang Quang Vu <vugomars@gmail.com> (c) 2022

# If you come from bash you might have to change your $PATH.
# export PATH=$HOME/bin:/usr/local/bin:$PATH

# Path to your oh-my-zsh installation.
export ZSH="$HOME/.oh-my-zsh"

# Themes
TYPEWRITTEN_RELATIVE_PATH="adaptive"
TYPEWRITTEN_CURSOR="underscore"

ZSH_THEME="typewritten/typewritten"

# Uncomment the following line to automatically update without prompting.
DISABLE_UPDATE_PROMPT="true"

# Uncomment the following line to change how often to auto-update (in days).
```

```
export UPDATE_ZSH_DAYS=3


# Some Plug-in
plugins=(rust repo ripgrep fzf fd repo git colored-man-pages colorize pip python brew

source $ZSH/oh-my-zsh.sh

# You may need to manually set your language environment
export LANG=en_US.UTF-8
export LSP_USE_PLISTS=true

source ~/env.sh

# fzf
[ -f ~/.fzf.zsh ] && source ~/.fzf.zsh

# Rust Enviroment
source "$HOME/.cargo/env"
export RUST_SRC_PATH=$HOME/.rustup/toolchains/stable-x86_64-unknown-linux-gnu/lib/rus
export PATH=$PATH:$HOME/.cargo/bin

# NVM
export NVM_DIR="$HOME/.nvm"
[ -s "/opt/homebrew/opt/nvm/nvm.sh" ] && \. "/opt/homebrew/opt/nvm/nvm.sh"  # This lo
[ -s "/opt/homebrew/opt/nvm/etc/bash_completion.d/nvm" ] && \. "/opt/homebrew/opt/nvm

# pkg-config
export PKG_CONFIG_PATH=/opt/homebrew/bin/pkg-config
```

2.4 Pseudo early-init

- This file will be loaded before the content of Doom's private `init.el`,
  I add some special stuff which I want to load very early.

- File này sẽ được load trước khi dữ liệu trong file init.el được khởi chạy,
  Tôi muốn thêm 1 vài stuff đặc biệt và trong file này.

```
;;; pseudo-early-init.el -*- coding: utf-8-unix; lexical-binding: t; -*-
```

2.4.1 Useful functions

- Trong file này chúng ta sẽ định nghĩa 1 vài function, một trong số chúng có thể dược sử dụng trong các file config. Nó có thể được sử dụng trong một vài gói package, nhưng vũ thich tự định nghĩa nó hơn.

```
;;; === Primitives ===

;; (+bool "someval") ;; ==> t
(defun +bool (val) (not (null val)))

;;; === Higher order functions ===

;; (+foldr (lambda (a b) (message "(%d + %d)" a b) (+ a b)) 0 '(1 2 3 4 5)) ;; ==> 15
;; (5 + 0) -> (4 + 5) -> (3 + 9) -> (2 + 12) --> (1 + 14)
(defun +foldr (fun acc seq)
  (if (null seq) acc
    (funcall fun (car seq) (+foldr fun acc (cdr seq)))))

;; (+foldl (lambda (a b) (message "(%d + %d)" a b) (+ a b)) 0 '(1 2 3 4 5)) ;; ==> 15
;; (0 + 1) -> (1 + 2) -> (3 + 3) -> (6 + 4) -> (10 + 5)
(defun +foldl (fun acc seq)
  (if (null seq) acc
    (+foldl fun (funcall fun acc (car seq)) (cdr seq))))

;; (+all '(83 88 t "txt")) ;; ==> t
(defun +all (seq)
  (+foldr (lambda (r l) (and r l)) t seq))

;; (+some '(nil nil "text" nil 2)) ;; ==> t
(defun +some (seq)
  (+bool (+foldr (lambda (r l) (or r l)) nil seq)))

;; (+filter 'stringp '("A" 2 "C" nil 3)) ;; ==> ("A" "C")
(defun +filter (fun seq)
  (when seq
    (let ((head (car seq))
          (tail (cdr seq)))
      (if (funcall fun head)
          (cons head (+filter fun tail))
        (+filter fun tail)))))
```

```
;; (+zip '(1 2 3 4) '(a b c d) '("A" "B" "C" "D")) ;; ==> ((1 a "A") (2 b "B") (3 c "
(defun +zip (&rest seqs)
  (if (null (car seqs)) nil
    (cons (mapcar #'car seqs)
          (apply #'+zip (mapcar #'cdr seqs)))))

;;; === Strings ===

;; (+str-join ", " '("foo" "10" "bar")) ;; ==> "foo, 10, bar"
(defun +str-join (sep seq)
  (+foldl (lambda (l r) (concat l sep r))
          (car seq) (cdr seq)))

;; (+str-split "foo, 10, bar" ", ") ;; ==> ("foo" "10" "bar")
(defun +str-split (str sep)
  (let ((s (string-search sep str)))
    (if s (cons (substring str 0 s)
                (+str-split (substring str (+ s (length sep))) sep))
      (list str))))

(defun +str-replace (old new s)
  "Replaces OLD with NEW in S."
  (replace-regexp-in-string (regexp-quote old) new s t t))

(defun +str-replace-all (replacements s)
  "REPLACEMENTS is a list of cons-cells. Each `car` is replaced with `cdr` in S."
  (replace-regexp-in-string (regexp-opt (mapcar 'car replacements))
                            (lambda (it) (cdr (assoc-string it replacements)))
                            s t t))

;;; === Files, IO ===

(defun +file-mime-type (file)
  "Get MIME type for FILE based on magic codes provided by the 'file' command.
Return a symbol of the MIME type, ex: `text/x-lisp', `text/plain',
`application/x-object', `application/octet-stream', etc."
  (let ((mime-type (shell-command-to-string (format "file --brief --mime-type %s" fil
    (intern (string-trim-right mime-type))))
```

```
(defun +file-name-incremental (filename)
  "Return an unique file name for FILENAME.
If \"file.ext\" exists, returns \"file-0.ext\"."
  (let* ((ext (file-name-extension filename))
         (dir (file-name-directory filename))
         (file (file-name-base filename))
         (filename-regex (concat "^" file "\\(?:-\\(?1:[[:digit:]]+\\)\\)?" (if ext (
         (last-file (car (last (directory-files dir nil filename-regex))))
         (last-file-num (when (and last-file (string-match filename-regex last-file)
         (num (1+ (string-to-number (or last-file-num "-1"))))
         (filename (file-name-concat dir (format "%s%s%s" file (if last-file (format
    filename))

(defun +file-read-to-string (filename)
  "Return a string with the contents of FILENAME."
  (when (and (file-exists-p filename) (not (file-directory-p filename)))
    (with-temp-buffer
      (insert-file-contents filename)
      (buffer-string))))

;;; === Systemd ===

(defun +systemd-running-p (service)
  "Check if the systemd SERVICE is running."
  (zerop (call-process "systemctl" nil nil nil "--user" "is-active" "--quiet" service

(defun +systemd-command (service command &optional pre-fn post-fn)
  "Call systemd with COMMAND and SERVICE."
  (interactive)
  (when pre-fn (funcall pre-fn))
  (let ((success (zerop (call-process "systemctl" nil nil nil "--user" command servic
    (unless success
      (message "[systemd]: Failed on calling '%s' on service %s.service." command ser
    (when post-fn (funcall post-fn success))
    success))

(defun +systemd-start (service &optional pre-fn post-fn)
  "Start systemd SERVICE."
  (interactive)
  (+systemd-command service "start" pre-fn post-fn))
```

```
(defun +systemd-stop (service &optional pre-fn post-fn)
  "Stops the systemd SERVICE."
  (interactive)
  (+systemd-command service "stop" pre-fn post-fn))
```

2.4.2 Fixes

```
;; Fixes to apply early

(when (daemonp)
  ;; When starting Emacs in daemon mode,
  ;; I need to have a valid passphrase in the gpg-agent.
  (let ((try-again 3)
        unlocked)
    (while (not (or unlocked (zerop try-again)))
      (setq unlocked (zerop (shell-command "gpg -q --no-tty --logger-file /dev/null -
            try-again (1- try-again))
      (unless unlocked
        (message "GPG: failed to unlock, please try again (%d)" try-again)))
    (unless unlocked ;; Exit Emacs, systemd will restart it
      (kill-emacs 1))))
```

2.4.3 Check for external tools

Some added packages require external tools, I like to check for these tools and store the result in global constants.

```
(defconst EAF-DIR (expand-file-name "eaf/eaf-repo" doom-data-dir))
(defconst IS-LUCID (string-search "LUCID" system-configuration-features))
;;(defconst FRICAS-DIR "/usr/lib/fricas/emacs")

(defconst AG-P (executable-find "ag"))
(defconst EAF-P (and (not IS-LUCID) (file-directory-p EAF-DIR)))
(defconst MPD-P (+all (mapcar #'executable-find '("mpc" "mpd"))))
(defconst MPV-P (executable-find "mpv"))
(defconst REPO-P (executable-find "repo"))
;;(defconst FRICAS-P (and (executable-find "fricas") (file-directory-p FRICAS-DIR)))
(defconst MAXIMA-P (executable-find "maxima"))
(defconst TUNTOX-P (executable-find "tuntox"))
(defconst ROSBAG-P (executable-find "rosbag"))
```

```
(defconst ZOTERO-P (executable-find "zotero"))
(defconst CHEZMOI-P (executable-find "chezmoi"))
(defconst STUNNEL-P (executable-find "stunnel"))
(defconst ECRYPTFS-P (+all (mapcar #'executable-find '("ecryptfs-add-passphrase" "/sb
(defconst BITWARDEN-P (executable-find "bw"))
(defconst YOUTUBE-DL-P (+some (mapcar #'executable-find '("yt-dlp" "youtube-dl"))))
(defconst NETEXTENDER-P (and (executable-find "netExtender") (+all (mapcar #'file-exi
(defconst CLANG-FORMAT-P (executable-find "clang-format"))
(defconst LANGUAGETOOL-P (executable-find "languagetool"))
```

2.5 Doom modules (`init.el`)

- Đây là file cấu hình sẵn của Doom, bạn chỉ việc bật những chức năng bạn cần lên `Doom` sẽ tự cài đặt cho bạn trong lần synchronization tiếp theo.

```
;;; init.el -*- coding: utf-8-unix; lexical-binding: t; -*-

;; This file controls what Doom modules are enabled and what order they load in.
;; Press 'K' on a module to view its documentation, and 'gd' to browse its directory.

;; I add some special stuff wich I want to load very early.
(load! "pseudo-early-init.el")

(doom! :input
       ;;chinese
       ;;japanese
       ;;layout            ; auie,ctsrnm is the superior home row

       :completion
       (company +childframe)           ; the ultimate code completion backend
       ;;helm              ; the *other* search engine for love and life
       ;;ido               ; the other *other* search engine...
       (ivy +childframe +icons)               ; a search engine for love and life
       ;; (vertico +icons)           ; the search engine of the future

       :ui
       deft              ; notational velocity for Emacs
       doom              ; what makes DOOM look the way it does
       doom-dashboard    ; a nifty splash screen for Emacs
```

```
;; doom-quit         ; DOOM quit-message prompts when you quit Emacs
(emoji +unicode +ascii +github)  ;
hl-todo           ; highlight TODO/FIXME/NOTE/DEPRECATED/HACK/REVIEW
hydra
indent-guides     ; highlighted indent columns
(ligatures +extra)         ; ligatures and symbols to make your code pretty ag
;;minimap           ; show a map of the code on the side
modeline          ; snazzy, Atom-inspired modeline, plus API
nav-flash         ; blink cursor line after big motions
;;neotree           ; a project drawer, like NERDTree for vim
ophints           ; highlight the region an operation acts on
(popup +all +defaults)   ; tame sudden yet inevitable temporary windows
;;tabs              ; a tab bar for Emacs
(treemacs +lsp)          ; a project drawer, like neotree but cooler
unicode           ; extended unicode support for various languages
(vc-gutter +diff-hl +pretty)         ; vcs diff in the fringe
vi-tilde-fringe   ; fringe tildes to mark beyond EOB
(window-select +numbers)     ; visually switch windows
workspaces        ; tab emulation, persistence & separate workspaces
zen               ; distraction-free coding or writing

:editor
(evil +everywhere); come to the dark side, we have cookies
file-templates    ; auto-snippets for empty files
fold              ; (nigh) universal code folding
(format +onsave)  ; automated prettiness
;;god               ; run Emacs commands without modifier keys
;;lispy             ; vim for lisp, for people who don't like vim
multiple-cursors  ; editing in many places at once
;;objed             ; text object editing for the innocent
;; parinfer          ; turn lisp into python, sort of
;;rotate-text       ; cycle region at point between text candidates
snippets          ; my elves. They type so I don't have to
word-wrap         ; soft wrapping with language-aware indent

:emacs
(dired +icons)             ; making dired pretty [functional]
;; electric          ; smarter, keyword-based electric-indent
(ibuffer +icons)         ; interactive buffer management
undo              ; persistent, smarter undo for your inevitable mistakes
```

```
vc                ; version-control and Emacs, sitting in a tree

:term
eshell            ; the elisp shell that works everywhere
shell             ; simple shell REPL for Emacs
term              ; basic terminal emulator for Emacs
vterm             ; the best terminal emulation in Emacs

:checkers
(syntax +childframe)              ; tasing you for every semicolon you forget
(spell +flyspell) ; tasing you for misspelling mispelling
(grammar +lsp)           ; tasing grammar mistake every you make

:tools
;;ansible
biblio            ; Writes a PhD for you (citation needed)
(debugger +lsp)          ; FIXME stepping through code, to help you add bugs
tree-sitter
direnv
(docker +lsp)
editorconfig      ; let someone else argue about tabs vs spaces
ein               ; tame Jupyter notebooks with emacs
(eval +overlay)     ; run code, run (also, repls)
gist              ; interacting with github gists
(lookup
  +dictionary
  +docsets)              ; navigate your code and its documentation
(lsp +peek)               ; M-x vscode
(magit +forge)              ; a git porcelain for Emacs
make              ; run make tasks from Emacs
;;pass              ; password manager for nerds
pdf               ; pdf enhancements
;;prodigy           ; FIXME managing external services & code builders
rgb               ; creating color strings
;;taskrunner        ; taskrunner for all your projects
;;terraform         ; infrastructure as code
tmux              ; an API for interacting with tmux
upload            ; map local to remote projects via ssh/ftp

:os
```

```
(:if IS-MAC macos)  ; improve compatibility with macOS
(tty +osc)               ; improve the terminal Emacs experience

:lang
;;agda              ; types of types of types of types...
;;beancount         ; mind the GAAP
(cc +lsp)                ; C > C++ == 1
;;clojure           ; java with a lisp
common-lisp       ; if you've seen one lisp, you've seen them all
;;coq               ; proofs-as-programs
;;crystal           ; ruby at the speed of c
;;csharp            ; unity, .NET, and mono shenanigans
data              ; config/data formats
;;(dart +flutter)   ; paint ui and not much else
;;dhall
;;elixir            ; erlang done right
;;elm               ; care for a cup of TEA?
emacs-lisp        ; drown in parentheses
;;erlang            ; an elegant language for a more civilized age
(ess +lsp)               ; emacs speaks statistics
;;factor
;;faust             ; dsp, but you get to keep your soul
;;fortran           ; in FORTRAN, GOD is REAL (unless declared INTEGER)
;;fsharp            ; ML stands for Microsoft's Language
;;fstar             ; (dependent) types and (monadic) effects and Z3
;;gdscript          ; the language you waited for
;;(go +lsp)         ; the hipster dialect
;;(haskell +lsp)    ; a language that's lazier than I am
;;hy                ; readability of scheme w/ speed of python
;;idris             ; a language you can depend on
(json +lsp +tree-sitter)              ; At least it ain't XML
;;(java +meghanada) ; the poster child for carpal tunnel syndrome
(javascript +lsp +tree-sitter)        ; all(hope(abandon(ye(who(enter(here))))
;;julia             ; a better, faster MATLAB
;;kotlin            ; a better, slicker Java(Script)
(latex
 +latexmk
 +cdlatex
 +lsp
 +fold)            ; writing papers in Emacs has never been so fun
```

```
;;lean               ; for folks with too much to prove
;;ledger             ; be audit you can be
(lua +lsp)               ; one-based indices? one-based indices
(markdown +grip)          ; writing docs for people to ignore
;;nim                ; python + lisp at the speed of c
nix                ; I hereby declare "nix geht mehr!"
;;ocaml              ; an objective camel
(org
 +attach
 +babel
 +capture
 +dragndrop
 +hugo
 ;; +jupyter
 +export
 +pandoc
 +gnuplot
 +pretty
 +present
 +protocol
 +pomodoro
 +roam2
 +noter)               ; organize your plain life in plain text
(php +lsp)               ; perl's insecure younger brother
plantuml           ; diagrams for confusing people more
;;purescript         ; javascript, but functional
(python +lsp +pyright)            ; beautiful is better than ugly
qt                 ; the 'cutest' gui framework ever
;;racket             ; a DSL for DSLs
;;raku               ; the artist formerly known as perl6
rest               ; Emacs as a REST client
;;rst                ; ReST in peace
(ruby +rails +lsp)     ; 1.step {|i| p "Ruby is #{i.even? ? 'love' : 'life'}"}
(rust +lsp +tree-sitter)              ; Fe2O3.unwrap().unwrap().unwrap().unwra
;;scala              ; java, but good
(scheme +guile +chez +mit +chicken +gauche +chibi)   ; a fully conniving famil
(sh +lsp +powershell)                ; she sells {ba,z,fi}sh shells on the C x
;;sml
solidity           ; do you need a blockchain? No.
;;swift              ; who asked for emoji variables?
```

```
;;terra             ; Earth and Moon in alignment for performance.
(web +lsp +tree-sitter)               ; the tubes
;;(toml +prettier-plugin-toml)
(yaml +lsp)             ; JSON, but readable
;;zig               ; C, but simpler

:email
(mu4e +org +gmail)
;;notmuch
;;(wanderlust +gmail)

:app
calendar
emms
everywhere        ; *leave* Emacs!? You must be joking
irc               ; how neckbeards socialize
;;(rss +org)        ; emacs as an RSS reader
twitter           ; twitter client https://twitter.com/vnought

:config
literate
(default +bindings +smartparens)

:private
;;(grammar +lsp)
;;(binary +disasm)
(dired-ng +icons +bindings)
)
```

2.6 Additional packages (`packages.el`)

- This file shouldn't be byte compiled.

```
;; -*- coding: utf-8-unix; no-byte-compile: t; -*-
```

3 General Emacs Setting

3.1 System/User/Default information

- sử dụng `setq` để set giá trị. (setq [SYM VAL]) với SYM là symbol `literal`, còn VAl là `evaluated`.

- Sử dụng `setq-default` để set giá trị mặc định.
- `defvar` tạo biến. (defvar SYMBOL &optional INITVALUE DOCSTRING)
- `defadvice` tại hàm (function).

```
;; global mode
(global-subword-mode 1)

;;
(setq user-full-name "Dang Quang Vu"
      user-mail-address "vugomars@gmail.com"
      user-blog-url "https://www.vugomars.com"

      ;; Split horizontally to right, vertically below the current window.
      evil-vsplit-window-right t
      evil-split-window-below t


      ispell-program-name "/usr/local/bin/aspell"
      racer-rust-src-path "~/.rustup/toolchains/stable-aarch64-apple-darwin/lib/rustl
      racer-rust-src-path "~/.rustup/toolchains/nightly-aarch64-apple-darwin/lib/rust
      parinfer-rust-library "~/.emacs.d/.local/etc/parinfer-rust"

      ;; Enable relative line number
      display-line-numbers-type 'relative

      source-directory (expand-file-name "~/.emacs.d/")


      )

(setq-default delete-by-moving-to-trash t ;; Delete files by moving them to trash.
              trash-directory nil

              ;; Take new window space from all other windows
              window-combination-resize t
              ;; stretch cursor to the glyph width
              x-stretch-cursor t
```

```
              )
```

3.2 Common Variables

```
(defvar +my/lang-main          "en")
(defvar +my/lang-secondary     "cn")
(defvar +my/lang-mother-tongue "vn")

(defvar +my/biblio-libraries-list (list (expand-file-name "~/Zotero/library.bib")))
(defvar +my/biblio-storage-list   (list (expand-file-name "~/Zotero/storage/")))
(defvar +my/biblio-notes-path     (expand-file-name "~/Dropbox/PhD/bibliography/notes
(defvar +my/biblio-styles-path    (expand-file-name "~/Zotero/styles/"))

;; Set it early, to avoid creating "~/org" at startup
(setq org-directory "~/Dropbox/Org")
```

3.3 Common Function

3.3.1 Show list of buffer when splitting

```
(defadvice! prompt-for-buffer (&rest _)
  :after '(evil-window-split evil-window-vsplit)
  (consult-buffer))
```

3.3.2 mine

```
;;;###autoload
(defun dqv/edit-zsh-configuration ()
  (interactive)
  (find-file "~/.zshrc"))

;;;###autoload
(defun dqv/use-eslint-from-node-modules ()
    "Set local eslint if available."
    (let* ((root (locate-dominating-file
                  (or (buffer-file-name) default-directory)
                  "node_modules"))
```

```
            (eslint (and root
                         (expand-file-name "node_modules/eslint/bin/eslint.js"
                                           root))))
      (when (and eslint (file-executable-p eslint))
        (setq-local flycheck-javascript-eslint-executable eslint))))

;;;###autoload
(defun dqv/goto-match-paren (arg)
  "Go to the matching if on (){}[], similar to vi style of % ."
  (interactive "p")
  (cond ((looking-at "[\[\(\{]") (evil-jump-item))
        ((looking-back "[\]\)\}]" 1) (evil-jump-item))
        ((looking-at "[\]\)\}]") (forward-char) (evil-jump-item))
        ((looking-back "[\[\(\{]" 1) (backward-char) (evil-jump-item))
        (t nil)))

;;;###autoload
(defun dqv/string-inflection-cycle-auto ()
  "switching by major-mode"
  (interactive)
  (cond
   ;; for emacs-lisp-mode
   ((eq major-mode 'emacs-lisp-mode)
    (string-inflection-all-cycle))
   ;; for python
   ((eq major-mode 'python-mode)
    (string-inflection-python-style-cycle))
   ;; for java
   ((eq major-mode 'java-mode)
    (string-inflection-java-style-cycle))
   (t
    ;; default
    (string-inflection-all-cycle))))

;; Current time and date
(defvar current-date-time-format "%Y-%m-%d %H:%M:%S"
  "Format of date to insert with `insert-current-date-time' func
See help of `format-time-string' for possible replacements")

(defvar current-time-format "%H:%M:%S"
```

```
  "Format of date to insert with `insert-current-time' func.
Note the weekly scope of the command's precision.")

;;;###autoload
(defun insert-current-date-time ()
  "insert the current date and time into current buffer.
Uses `current-date-time-format' for the formatting the date/time."
  (interactive)
  (insert (format-time-string current-date-time-format (current-time)))
  )

;;;###autoload
(defun insert-current-time ()
  "insert the current time (1-week scope) into the current buffer."
  (interactive)
  (insert (format-time-string current-time-format (current-time)))
  )

;;;###autoload
(defun my/capitalize-first-char (&optional string)
  "Capitalize only the first character of the input STRING."
  (when (and string (> (length string) 0))
    (let ((first-char (substring string nil 1))
          (rest-str   (substring string 1)))
      (concat (capitalize first-char) rest-str))))

;;;###autoload
(defun my/lowcase-first-char (&optional string)
  "Capitalize only the first character of the input STRING."
  (when (and string (> (length string) 0))
    (let ((first-char (substring string nil 1))
          (rest-str   (substring string 1)))
      (concat first-char rest-str))))

;;;###autoload
(defun dqv/async-shell-command-silently (command)
  "async shell command silently."
  (interactive)
  (let
      ((display-buffer-alist
```

```
        (list
         (cons
          "\\*Async Shell Command\\*.*"
          (cons #'display-buffer-no-window nil)))))
    (async-shell-command
     command)))
```

3.3.3 Scroll page

```
(defun scroll-half-page-down ()
  "scroll down half the page"
  (interactive)
  (scroll-down (/ (window-body-height) 2)))

(defun scroll-half-page-up ()
  "scroll up half the page"
  (interactive)
  (scroll-up (/ (window-body-height) 2)))
```

3.4 Undo

3.4.1 Undo-fu

Tweak `undo-fu` and other stuff from Doom's `:emacs undo`.

```
;; Increase undo history limits even more
(after! undo-fu
  (setq undo-limit        10000000     ;; 1MB   (default is 160kB, Doom's default is
        undo-strong-limit 100000000    ;; 100MB (default is 240kB, Doom's default is
        undo-outer-limit  1000000000)) ;; 1GB   (default is 24MB,  Doom's default is

(after! evil
  (setq evil-want-fine-undo t)) ;; By default while in insert all changes are one big
```

3.4.2 Visual undo (`vundo`)

- Visualize Undo tree

```
(package! vundo
  :recipe (:host github
           :repo "casouri/vundo")
  :pin "16a09774ddfbd120d625cdd35fcf480e76e278bb")
```

```
(use-package! vundo
  :defer t
  :init
  (defconst +vundo-unicode-symbols
   '((selected-node   . ? )
     (node            . ? )
     (vertical-stem   . ? )
     (branch          . ? )
     (last-branch     . ? )
     (horizontal-stem . ? )))

  (map! :leader
        (:prefix ("o")
         :desc "vundo" "v" #'vundo))

  :config
  (setq vundo-glyph-alist +vundo-unicode-symbols
        vundo-compact-display t
        vundo-window-max-height 6))
```

3.5 Messages buffer

Stick to buffer tail, useful with `*Messages*` buffer. Derived from this answer.

```
(defvar +messages--auto-tail-enabled nil)

(defun +messages--auto-tail-a (&rest arg)
  "Make *Messages* buffer auto-scroll to the end after each message."
  (let* ((buf-name (buffer-name (messages-buffer)))
         ;; Create *Messages* buffer if it does not exist
         (buf (get-buffer-create buf-name)))
    ;; Activate this advice only if the point is _not_ in the *Messages* buffer
    ;; to begin with. This condition is required; otherwise you will not be
    ;; able to use `isearch' and other stuff within the *Messages* buffer as
    ;; the point will keep moving to the end of buffer :P
    (when (not (string= buf-name (buffer-name)))
      ;; Go to the end of buffer in all *Messages* buffer windows that are
      ;; *live* (`get-buffer-window-list' returns a list of only live windows).
      (dolist (win (get-buffer-window-list buf-name nil :all-frames))
        (with-selected-window win
```

```
          (goto-char (point-max))))
      ;; Go to the end of the *Messages* buffer even if it is not in one of
      ;; the live windows.
      (with-current-buffer buf
        (goto-char (point-max))))))

(defun +messages-auto-tail-toggle ()
  "Auto tail the '*Messages*' buffer."
  (interactive)
  (if +messages--auto-tail-enabled
      (progn
        (advice-remove 'message '+messages--auto-tail-a)
        (setq +messages--auto-tail-enabled nil)
        (message "+messages-auto-tail: Disabled."))
    (advice-add 'message :after '+messages--auto-tail-a)
    (setq +messages--auto-tail-enabled t)
    (message "+messages-auto-tail: Enabled.")))
```

3.6 Secrets

Set the path to my GPG encrypted secrets. I like to set the cache expiry to `nil` instead of the default 2 hours.

```
(setq auth-sources '("~/.authinfo.gpg")
      auth-source-do-cache t
      auth-source-cache-expiry 86400 ; All day, defaut is 2h (7200)
      password-cache t
      password-cache-expiry 86400)

(after! epa
  (setq-default epa-file-encrypt-to '("F2204E74F9D848C2")))
```

4 Emacs daemon

4.1 Initialization

```
(defun +daemon-startup ()
  ;; mu4e
  (when (require 'mu4e nil t)
    ;; Automatically start `mu4e' in background.
```

```
    (when (load! "mu-lock.el" (expand-file-name "email/mu4e/autoload" doom-modules-di
      (setq +mu4e-lock-greedy t
            +mu4e-lock-relaxed t)
      (when (+mu4e-lock-available t)
        ;; Check each 5m, if `mu4e' if closed, start it in background.
        (run-at-time nil ;; Launch now
                     (* 60 5) ;; Check each 5 minutes
                     (lambda ()
                       (when (and (not (mu4e-running-p)) (+mu4e-lock-available))
                         (mu4e--start)
                         (message "Started `mu4e' in background.")))))))

  ;; RSS
  (when (require 'elfeed nil t)
    (run-at-time nil (* 2 60 60) #'elfeed-update))) ;; Check every 2h

(when (daemonp)
  ;; At daemon startup
  (add-hook 'emacs-startup-hook #'+daemon-startup)

  ;; After creating a new frame (via emacsclient)
  ;; Reload Doom's theme
  (add-hook 'server-after-make-frame-hook #'doom/reload-theme))
```

4.2 Tweaks

4.2.1 Save recent files

When editing files with Emacs client, the files does not get stored by `recentf`, making Emacs forgets about recently opened files. A quick fix is to hook the `recentf-save-list` command to the `delete-frame-functions` and `delete-terminal-functions` which gets executed each time a frame/terminal is deleted.

```
(when (daemonp)
  (add-hook! '(delete-frame-functions delete-terminal-functions)
    (let ((inhibit-message t))
      (recentf-save-list)
      (savehist-save))))
```

5 User Interface

5.1 Font

```
(setq doom-font (font-spec :family "Iosevka Fixed Curly Slab" :size 15)
      doom-big-font (font-spec :family "Iosevka Fixed Curly Slab" :size 20 :weight 'l
      doom-variable-pitch-font (font-spec :family "Iosevka Fixed Curly Slab")
      doom-unicode-font (font-spec :family "JuliaMono")
      doom-serif-font (font-spec :family "Iosevka Fixed Curly Slab" :weight 'light))
```

5.2 Theme

5.2.1 Doom

```
;; (setq doom-theme 'doom-one-light)
;; (remove-hook 'window-setup-hook #'doom-init-theme-h)
;; (add-hook 'after-init-hook #'doom-init-theme-h 'append)
```

5.2.2 Modus themes

```
(package! modus-themes)
```

```
(use-package! modus-themes
  :init
  (setq modus-themes-hl-line '(accented intense)
        modus-themes-subtle-line-numbers t
        modus-themes-region '(bg-only no-extend) ;; accented
        modus-themes-variable-pitch-ui nil
        modus-themes-fringes 'subtle
        modus-themes-diffs nil
        modus-themes-italic-constructs t
        modus-themes-bold-constructs t
        modus-themes-intense-mouseovers t
        modus-themes-paren-match '(bold intense)
        modus-themes-syntax '(green-strings)
        modus-themes-links '(neutral-underline background)
        modus-themes-mode-line '(borderless padded)
        modus-themes-tabs-accented nil ;; default
        modus-themes-completions
        '((matches . (extrabold intense accented))
          (selection . (semibold accented intense))
          (popup . (accented)))
```

```
      modus-themes-headings '((1 . (rainbow 1.4))
                              (2 . (rainbow 1.3))
                              (3 . (rainbow 1.2))
                              (4 . (rainbow bold 1.1))
                              (t . (rainbow bold)))
      modus-themes-org-blocks 'gray-background
      modus-themes-org-agenda
      '((header-block . (semibold 1.4))
        (header-date . (workaholic bold-today 1.2))
        (event . (accented italic varied))
        (scheduled . rainbow)
        (habit . traffic-light))
      modus-themes-markup '(intense background)
      modus-themes-mail-citations 'intense
      modus-themes-lang-checkers '(background))

(defun +modus-themes-tweak-packages ()
  (modus-themes-with-colors
    (set-face-attribute 'cursor nil :background (modus-themes-color 'blue))
    (set-face-attribute 'font-lock-type-face nil :foreground (modus-themes-color 'm
    (custom-set-faces
     ;; Tweak `evil-mc-mode'
     `(evil-mc-cursor-default-face ((,class :background ,magenta-intense-bg)))
     ;; Tweak `git-gutter-mode'
     `(git-gutter-fr:added ((,class :foreground ,green-fringe-bg)))
     `(git-gutter-fr:deleted ((,class :foreground ,red-fringe-bg)))
     `(git-gutter-fr:modified ((,class :foreground ,yellow-fringe-bg)))
     ;; Tweak `doom-modeline'
     `(doom-modeline-evil-normal-state ((,class :foreground ,green-alt-other)))
     `(doom-modeline-evil-insert-state ((,class :foreground ,red-alt-other)))
     `(doom-modeline-evil-visual-state ((,class :foreground ,magenta-alt)))
     `(doom-modeline-evil-operator-state ((,class :foreground ,blue-alt)))
     `(doom-modeline-evil-motion-state ((,class :foreground ,blue-alt-other)))
     `(doom-modeline-evil-replace-state ((,class :foreground ,yellow-alt)))
     ;; Tweak `diff-hl-mode'
     `(diff-hl-insert ((,class :foreground ,green-fringe-bg)))
     `(diff-hl-delete ((,class :foreground ,red-fringe-bg)))
     `(diff-hl-change ((,class :foreground ,yellow-fringe-bg)))
     ;; Tweak `solaire-mode'
     `(solaire-default-face ((,class :inherit default :background ,bg-alt :foregrou
```

```
      `(solaire-line-number-face ((,class :inherit solaire-default-face :foreground
      `(solaire-hl-line-face ((,class :background ,bg-active)))
      `(solaire-org-hide-face ((,class :background ,bg-alt :foreground ,bg-alt)))
      ;; Tweak `display-fill-column-indicator-mode'
      `(fill-column-indicator ((,class :height 0.3 :background ,bg-inactive :foregro
      ;; Tweak `mmm-mode'
      `(mmm-cleanup-submode-face ((,class :background ,yellow-refine-bg)))
      `(mmm-code-submode-face ((,class :background ,bg-active)))
      `(mmm-comment-submode-face ((,class :background ,blue-refine-bg)))
      `(mmm-declaration-submode-face ((,class :background ,cyan-refine-bg)))
      `(mmm-default-submode-face ((,class :background ,bg-alt)))
      `(mmm-init-submode-face ((,class :background ,magenta-refine-bg)))
      `(mmm-output-submode-face ((,class :background ,red-refine-bg)))
      `(mmm-special-submode-face ((,class :background ,green-refine-bg))))))

  (add-hook 'modus-themes-after-load-theme-hook #'+modus-themes-tweak-packages)

  :config
  (modus-themes-load-operandi)
  (map! :leader
        :prefix "t" ;; toggle
        :desc "Toggle Modus theme" "m" #'modus-themes-toggle))
```

5.3 Mode line

Clock

- Display time and set the format to 24h.

```
(after! doom-modeline
  (setq display-time-string-forms
        '((propertize (concat "   " 24-hours ":" minutes))))
  (display-time-mode 1) ; Enable time in the mode-line

  ;; Add padding to the right
  (doom-modeline-def-modeline 'main
   '(bar workspace-name window-number modals matches follow buffer-info remote-host b
   '(objed-state misc-info persp-name battery grip irc mu4e gnus github debug repl ls
```

Battery

- Show battery level unless battery is not present or battery information is unknown.

```
(after! doom-modeline
  (let ((battery-str (battery)))
    (unless (or (equal "Battery status not available" battery-str)
                (string-match-p (regexp-quote "unknown") battery-str)
                (string-match-p (regexp-quote "N/A") battery-str))
      (display-battery-mode 1))))
```

Mode line customization

```
(after! doom-modeline
  (setq doom-modeline-bar-width 4
        doom-modeline-mu4e t
        doom-modeline-major-mode-icon t
        doom-modeline-major-mode-color-icon t
        doom-modeline-buffer-file-name-style 'truncate-upto-project))
```

5.4 Set transparency & Full Screen

```
;; set transparent
(set-frame-parameter (selected-frame) 'alpha '(97 100))
(add-to-list 'default-frame-alist '(alpha 97 100))

;; full screen
(add-to-list 'initial-frame-alist '(fullscreen . maximized))
(add-hook 'org-mode-hook 'turn-on-auto-fill)
```

5.5 Dashboard

Custom splash image Change the logo to an image, a set of beautiful images can be found in `assets`.

```
(setq fancy-splash-image (expand-file-name "assets/doom-emacs-gray.svg" doom-user-dir
```

Dashboard

```
(remove-hook '+doom-dashboard-functions #'doom-dashboard-widget-shortmenu)
(remove-hook '+doom-dashboard-functions #'doom-dashboard-widget-footer)
(add-hook! '+doom-dashboard-mode-hook (hl-line-mode -1))
(setq-hook! '+doom-dashboard-mode-hook evil-normal-state-cursor (list nil))
```

5.6 SVG tag and `svg-lib`

```
(package! svg-tag-mode :pin "efd22edf650fb25e665269ba9fed7ccad0771a2f")
```

```
(use-package! svg-tag-mode
  :commands svg-tag-mode
  :config
  (setq svg-tag-tags
        '(("^\\*.* .* \\(:[A-Za-z0-9]+\\)" .
           ((lambda (tag)
              (svg-tag-make
               tag
               :beg 1
               :font-family "Roboto Mono"
               :font-size 10
               :height 0.8
               :padding 0
               :margin 0))))
          ("\\(:[A-Za-z0-9]+:\\)$" .
           ((lambda (tag)
              (svg-tag-make
               tag
               :beg 1
               :end -1
               :font-family "Roboto Mono"
               :font-size 10
               :height 0.8
               :padding 0
               :margin 0)))))))
```

```
(after! svg-lib
  ;; Set `svg-lib' cache directory
  (setq svg-lib-icons-dir (expand-file-name "svg-lib" doom-data-dir)))
```

5.7 Focus

Dim the font color of text in surrounding paragraphs, focus only on the current line.

```
(package! focus :pin "9dd85fc474bbc1ebf22c287752c960394fcd465a")
```

```
(use-package! focus
  :commands focus-mode)
```

5.8 Scrolling

```
(package! good-scroll
  :disable EMACS29+
  :pin "a7ffd5c0e5935cebd545a0570f64949077f71ee3")

(use-package! good-scroll
  :unless EMACS29+
  :config (good-scroll-mode 1))

(when EMACS29+
  (pixel-scroll-precision-mode 1))

(setq hscroll-step 1
      hscroll-margin 0
      scroll-step 1
      scroll-margin 0
      scroll-conservatively 101
      scroll-up-aggressively 0.01
      scroll-down-aggressively 0.01
      scroll-preserve-screen-position 'always
      auto-window-vscroll nil
      fast-but-imprecise-scrolling nil)
```

5.9 All the icons

Set some custom icons for some file extensions, basically for `.m` files.

```
(after! all-the-icons
  (setcdr (assoc "m" all-the-icons-extension-icon-alist)
          (cdr (assoc "matlab" all-the-icons-extension-icon-alist))))
```

5.10 Tabs

```
(after! centaur-tabs
  ;; For some reason, setting `centaur-tabs-set-bar' this to `right'
  ;; instead of Doom's default `left', fixes this issue with Emacs daemon:
  ;; https://github.com/doomemacs/doomemacs/issues/6647#issuecomment-1229365473
  (setq centaur-tabs-set-bar 'right
        centaur-tabs-gray-out-icons 'buffer
        centaur-tabs-set-modified-marker t
```

```
        centaur-tabs-close-button " "
        centaur-tabs-modified-marker " "))
```

5.11 Zen (writeroom) mode

```
(after! writeroom-mode
  ;; Show mode line
  (setq writeroom-mode-line t)

  ;; Disable line numbers
  (add-hook! 'writeroom-mode-enable-hook
    (when (bound-and-true-p display-line-numbers-mode)
      (setq-local +line-num--was-activate-p display-line-numbers-type)
      (display-line-numbers-mode -1)))

  (add-hook! 'writeroom-mode-disable-hook
    (when (bound-and-true-p +line-num--was-activate-p)
      (display-line-numbers-mode +line-num--was-activate-p)))

  (after! org
    ;; Increase latex previews scale in Zen mode
    (add-hook! 'writeroom-mode-enable-hook (+org-format-latex-set-scale 2.0))
    (add-hook! 'writeroom-mode-disable-hook (+org-format-latex-set-scale 1.4)))

  (after! blamer
    ;; Disable blamer in zen (writeroom) mode
    (add-hook! 'writeroom-mode-enable-hook
      (when (bound-and-true-p blamer-mode)
        (setq +blamer-mode--was-active-p t)
        (blamer-mode -1)))
    (add-hook! 'writeroom-mode-disable-hook
      (when (bound-and-true-p +blamer-mode--was-active-p)
        (blamer-mode 1)))))
```

5.12 Highlight indent guides

```
(after! highlight-indent-guides
  (setq highlight-indent-guides-character ?
        highlight-indent-guides-responsive 'top))
```

6 Keybindings

Keybindings reference: evil-bindings.el editor/evil/config.el

6.1 Disable keybindings

```
(global-set-key (kbd "<f1>") nil)        ; ns-print-buffer
(global-set-key (kbd "<f2>") nil)        ; ns-print-buffer
(define-key evil-normal-state-map (kbd ",") nil)
(define-key evil-visual-state-map (kbd ",") nil)
```

6.2 F1~12(kbd)

```
(global-set-key (kbd "<f1>") 'dqv-everything/body)
(global-set-key (kbd "<f2>") 'rgrep)
(global-set-key (kbd "<f5>") 'deadgrep)
(global-set-key (kbd "<M-f5>") 'deadgrep-kill-all-buffers)
;; (global-set-key (kbd "<f8>") 'quickrun)
(global-set-key (kbd "<f12>") 'smerge-vc-next-conflict)
(global-set-key (kbd "<S-f12>") '+vc/smerge-hydra/body)
(global-set-key (kbd "M-z") 'zzz-to-char)
;; (global-set-key (kbd "C-t") '+vterm/toggle)
;; (global-set-key (kbd "C-S-t") '+vterm/here)
;; (global-set-key (kbd "C-d") 'kill-current-buffer)
;; (avy-setup-default)
;; (global-set-key (kbd "C-c C-j") 'avy-resume)
```

6.3 Common

```
(setq doom-localleader-key ",")
(map!
;; avy
:nv    "f"     #'avy-goto-char-2
:nv    "F"     #'avy-goto-char
:nv    "w"     #'avy-goto-word-1
:nv    "W"     #'avy-goto-word-0

;; view scroll mode
:nv    "C-n"   #'scroll-half-page-up
:nv    "C-p"   #'scroll-half-page-down
```

```
:nv    "-"     #'evil-window-decrease-width
:nv    "+"     #'evil-window-increase-width
:nv    "C--"   #'evil-window-decrease-height
:nv    "C-+"   #'evil-window-increase-height

:nv    ")"     #'sp-forward-sexp
:nv    "("     #'sp-backward-up-sexp
:nv    "s-)"   #'sp-down-sexp
:nv    "s-("   #'sp-backward-sexp
:nv    "gd"    #'xref-find-definitions
:nv    "gD"    #'xref-find-references
:nv    "gb"    #'xref-pop-marker-stack

:niv   "C-e"   #'evil-end-of-line
:niv   "C-="   #'er/expand-region

"C-;"          #'tiny-expand
"C-a"          #'crux-move-beginning-of-line
"C-s"          #'+default/search-buffer

"C-c C-j"      #'avy-resume
"C-c c x"      #'org-capture
;; "C-c c j"      #'avy-resume

"C-c f r"      #'dqv/indent-org-block-automatically

"C-c h h"      #'dqv/org-html-export-to-html

"C-c i d"      #'insert-current-date-time
"C-c i t"      #'insert-current-time
;; "C-c i d"      #'crux-insert-date
"C-c i e"      #'emojify-inert-emoji
"C-c i f"      #'js-doc-insert-function-doc
"C-c i F"      #'js-doc-insert-file-doc

"C-c o o"      #'crux-open-with
"C-c o u"      #'crux-view-url
"C-c o t"      #'crux-visit-term-buffer
;; org-roam
"C-c o r o"    #'org-roam-ui-open
```

```
"C-c r r"      #'vr/replace
"C-c r q"      #'vr/query-replace

"C-c y y"      #'youdao-dictionary-search-at-point+

;; Command/Window
"s-k"          #'move-text-up
"s-j"          #'move-text-down
"s-i"          #'dqv/string-inflection-cycle-auto
;; "s--"          #'sp-splice-sexp
;; "s-_"          #'sp-rewrap-sexp

"M-i"          #'parrot-rotate-next-word-at-point
"M--"          #'dqv/goto-match-paren
)
```

6.4 SPC leader

```
(map! :leader
      :n "SPC"  #'execute-extended-command
      :n "."  #'dired-jump
      :n ","  #'magit-status
      :n "-"  #'goto-line
      ;; (:prefix ("d" . "Debugger")
      ;;  :n    "r"   #'dap-debug
      ;;  :n    "l"   #'dap-debug-last
      ;;  :n    "R"   #'dap-debug-recent
      ;;  :n    "x"   #'dap-disconnect
      ;;  :n    "a"   #'dap-breakpoint-add
      ;;  :n    "t"   #'dap-breakpoint-toggle
      ;;  :n    "d"   #'dap-delete-session
      ;;  :n    "D"   #'dap-delete-all-sessions

      ;;  )

      (:prefix ("e" . "Exercise Coding Challenger")
       :n    "l"     #'leetcode
       :n    "d"     #'leetcode-daily
       :n    "o"     #'leetcode-show-problem-in-browser
```

```
        :n    "s"     #'leetcode-show-problem
        )


       (:prefix ("m" . "Treemacs")
        :n     "t"           #'treemacs
        :n     "df"           #'treemacs-delete-file
        :n     "dp"           #'treemacs-remove-project-from-workspace
        :n     "cd"           #'treemacs-create-dir
        :n     "cf"           #'treemacs-create-file
        :n     "a"           #'treemacs-add-project-to-workspace
        :n     "wc"           #'treemacs-create-workspace
        :n     "ws"           #'treemacs-switch-workspace
        :n     "wd"           #'treemacs-remove-workspace
        :n     "wf"           #'treemacs-rename-workspace
        )

       :nv "w -" #'evil-window-split
       :nv "j" #'switch-to-buffer
       :nv "wo" #'delete-other-windows
       :nv "fd" #'doom/delete-this-file
       :nv "ls" #'+lsp/switch-client
       :nv "bR" #'rename-buffer
       :nv "bx" #'doom/switch-to-scratch-buffer
       )

(map! :map dap-mode-map
      :leader
      :prefix ("d" . "dap")
      ;; basics
      :desc "dap next"          "n" #'dap-next
      :desc "dap step in"       "i" #'dap-step-in
      :desc "dap step out"      "o" #'dap-step-out
      :desc "dap continue"      "c" #'dap-continue
      :desc "dap hydra"         "h" #'dap-hydra
      :desc "dap debug restart" "r" #'dap-debug-restart
      :desc "dap debug"         "s" #'dap-debug

      ;; debug
      :prefix ("dd" . "Debug")
```

```
      :desc "dap debug recent"  "r" #'dap-debug-recent
      :desc "dap debug last"    "l" #'dap-debug-last

      ;; eval
      :prefix ("de" . "Eval")
      :desc "eval"                "e" #'dap-eval
      :desc "eval region"         "r" #'dap-eval-region
      :desc "eval thing at point" "s" #'dap-eval-thing-at-point
      :desc "add expression"      "a" #'dap-ui-expressions-add
      :desc "remove expression"   "d" #'dap-ui-expressions-remove

      :prefix ("db" . "Breakpoint")
      :desc "dap breakpoint toggle"      "b" #'dap-breakpoint-toggle
      :desc "dap breakpoint condition"   "c" #'dap-breakpoint-condition
      :desc "dap breakpoint hit count"   "h" #'dap-breakpoint-hit-condition
      :desc "dap breakpoint log message" "l" #'dap-breakpoint-log-message)
```

6.5 org-mode

```
(map! :map org-mode-map
      ;; t
      :nv "tt"          #'org-todo
      :nv "tT"          #'counsel-org-tag

      :nv "tcc"         #'org-toggle-checkbox
      :nv "tcu"         #'org-update-checkbox-count

      :nv "tpp"         #'org-priority
      :nv "tpu"         #'org-priority-up
      :nv "tpd"         #'org-priority-down


      ;; C-c
      "C-c a t" #'org-transclusion-add
      ;; #'org-transclusion-mode
      "C-c c i" #'org-clock-in
      "C-c c o" #'org-clock-out
      "C-c c h" #'counsel-org-clock-history
      "C-c c g" #'counsel-org-clock-goto
      "C-c c c" #'counsel-org-clock-context
```

```
"C-c c r" #'counsel-org-clock-rebuild-history
"C-c c p" #'org-preview-html-mode

"C-c f r" #'dqv/indent-org-block-automatically

"C-c e e" #'all-the-icons-insert
"C-c e a" #'all-the-icons-insert-faicon
"C-c e f" #'all-the-icons-insert-fileicon
"C-c e w" #'all-the-icons-insert-wicon
"C-c e o" #'all-the-icons-insert-octicon
"C-c e m" #'all-the-icons-insert-material
"C-c e i" #'all-the-icons-insert-alltheicon

"C-c g l" #'org-mac-grab-link

"C-c i u" #'org-mac-chrome-insert-frontmost-url
"C-c i c" #'copyright
"C-c i D" #'o-docs-insert

;; `C-c s' links & search-engine
"C-c l l" #'org-super-links-link
"C-c l L" #'org-super-links-insert-link
"C-c l s" #'org-super-links-store-link
"C-c l d" #'org-super-links-quick-insert-drawer-link
"C-c l i" #'org-super-links-quick-insert-inline-link
"C-c l D" #'org-super-links-delete-link
"C-c l b" #'org-mark-ring-goto

"C-c q s" #'org-ql-search
"C-c q v" #'org-ql-view
"C-c q b" #'org-ql-sidebar
"C-c q r" #'org-ql-view-recent-items
"C-c q t" #'org-ql-sparse-tree

"C-c r f" #'org-refile-copy ;; copy current entry to another heading
"C-c r F" #'org-refile ;; like `org-refile-copy' but moving

"C-c w m" #'org-mind-map-write
"C-c w M" #'org-mind-map-write-current-tree
```

```
      ;; org-roam, org-ref
      ;; "C-c n l" #'org-roam-buffer-toggle
      ;; "C-c n f" #'org-roam-node-find
      ;; "C-c n g" #'org-roam-graph
      ;; "C-c n i" #'org-roam-node-insert
      ;; "C-c n c" #'org-roam-capture
      ;; "C-c n j" #'org-roam-dailies-capture-today
      "C-c n r a" #'org-roam-ref-add
      "C-c n r f" #'org-roam-ref-find
      "C-c n r d" #'org-roam-ref-remove
      "C-c n r c" #'org-ref-insert-cite-link
      "C-c n r l" #'org-ref-insert-label-link
      "C-c n r i" #'org-ref-insert-link
      "C-c n b c" #'org-bibtex-check-all
      "C-c n b a" #'org-bibtex-create
      )
```

6.6 deadgrep-mode

```
(map! :map deadgrep-mode-map
      :nv "TAB" #'deadgrep-toggle-file-results
      :nv "D"   #'deadgrep-directory
      :nv "S"   #'deadgrep-search-term
      :nv "N"   #'deadgrep--move-match
      :nv "n"   #'deadgrep--move
      :nv "o"   #'deadgrep-visit-result-other-window
      :nv "r"   #'deadgrep-restart)
```

7 Package Configuration

7.1 Editing

7.1.1 File templates

- Chúng ta có thể ghi đè một vài mode chung snippet's vào một thư mục.

```
(set-file-template! "\\.tex$" :trigger "__" :mode 'latex-mode)
(set-file-template! "\\.org$" :trigger "__" :mode 'org-mode)
(set-file-template! "/LICEN[CS]E$" :trigger '+file-templates/insert-license)
```

7.1.2 Scratch buffer

- Tell the scratch buffer to start in `emacs-lisp-mode`.

```
(setq doom-scratch-initial-major-mode 'emacs-lisp-mode)
```

7.1.3 Very large files

- Hỗ trợ tải file lớn với cách tải từng phần nhỏ, giúp sử dụng file lớn dễ dàng hơn.

```
(package! vlf :pin "cc02f2533782d6b9b628cec7e2dcf25b2d05a27c")
```

- Để sử dụng VLF mà không bị delay khi khởi động, chúng ta chỉ load nó trong im lặng under the hood.

```
(use-package! vlf-setup
  :defer-incrementally vlf-tune vlf-base vlf-write vlf-search vlf-occur vlf-follow vl
```

7.1.4 Evil

```
(after! evil
  ;; This fixes https://github.com/doomemacs/doomemacs/issues/6478
  ;; Ref: https://github.com/emacs-evil/evil/issues/1630
  (evil-select-search-module 'evil-search-module 'isearch)

  (setq evil-kill-on-visual-paste nil)) ; Don't put overwritten text in the kill ring
```

```
(package! evil-escape :disable t)
```

7.1.5 Aggressive indent

```
(package! aggressive-indent :pin "70b3f0add29faff41e480e82930a231d88ee9ca7")
```

```
(use-package! aggressive-indent
  :commands (aggressive-indent-mode))
```

7.1.6 YASnippet

Nested snippets are good, enable that.

```
(setq yas-triggers-in-field t)
```

7.2 Symbols

7.2.1 Emojify

- Cài đặt emoji của Twitter.

```
(setq emojify-emoji-set "twemoji-v2")
```

- Một vấn đề khi sử dụng emoji là nó sẽ ưu tiên sử dụng các ký hiệu mặc định, Điều này thương sảy ra với các overlay symbol trên org mode. Chúng ta sẽ xoá nó khỏi mục emoji.

```
(defvar emojify-disabled-emojis
  '(;; Org
    " " " " " " " " " " " " " " " " " " "®" " " " " " " " "
    ;; Terminal powerline
    " "
    ;; Box drawing
    " " " ")
  "Characters that should never be affected by `emojify-mode'.")

(defadvice! emojify-delete-from-data ()
  "Ensure `emojify-disabled-emojis' don't appear in `emojify-emojis'."
  :after #'emojify-set-emoji-data
  (dolist (emoji emojify-disabled-emojis)
    (remhash emoji emojify-emojis)))
```

- Chúng ta sẽ chạy một minor mode cho phép bạn nhập các biểu tượng emoji, và chuyển sang unicode.

```
(defun emojify--replace-text-with-emoji (orig-fn emoji text buffer start end &optiona
  "Modify `emojify--propertize-text-for-emoji' to replace ascii/github emoticons with
  (if (or (not emoticon-to-emoji) (= 1 (length text)))
      (funcall orig-fn emoji text buffer start end target)
    (delete-region start end)
    (insert (ht-get emoji "unicode"))))

(define-minor-mode emoticon-to-emoji
  "Write ascii/gh emojis, and have them converted to unicode live."
  :global nil
  :init-value nil
  (if emoticon-to-emoji
```

```
      (progn
        (setq-local emojify-emoji-styles '(ascii github unicode))
        (advice-add 'emojify--propertize-text-for-emoji :around #'emojify--replace-te
        (unless emojify-mode
          (emojify-turn-on-emojify-mode)))
    (setq-local emojify-emoji-styles (default-value 'emojify-emoji-styles))
    (advice-remove 'emojify--propertize-text-for-emoji #'emojify--replace-text-with-e
```

- Kết hợp chức năng này cho email & mu4e

```
(add-hook! '(mu4e-compose-mode org-msg-edit-mode circe-channel-mode) (emoticon-to-emo
```

7.2.2 Ligatures

- Vũ muốn vô hiệu hoá các extra ligature trên một vài mode (rust, c++, python, scheme). Thường chỉ giữ lại sử dụng cho emacs-lisp mode.

```
(defun +appened-to-negation-list (head tail)
  (if (sequencep head)
      (delete-dups
       (if (eq (car tail) 'not)
           (append head tail)
         (append tail head)))
    tail))

(when (modulep! :ui ligatures)
  (setq +ligatures-extras-in-modes
        (+appened-to-negation-list
         +ligatures-extras-in-modes
         '(not c-mode c++-mode emacs-lisp-mode python-mode scheme-mode racket-mode ru

  (setq +ligatures-in-modes
        (+appened-to-negation-list
         +ligatures-in-modes
         '(not emacs-lisp-mode scheme-mode racket-mode))))
```

7.3 Features

7.3.1 Workspaces

```
(map! :leader
```

```
(:when (modulep! :ui workspaces)
 :prefix ("TAB" . "workspace")
 :desc "Display tab bar"           "TAB" #'+workspace/display
 :desc "Switch workspace"          "."   #'+workspace/switch-to
 :desc "Switch to last workspace"  "$"   #'+workspace/other ;; Modified
 :desc "New workspace"             "n"   #'+workspace/new
 :desc "New named workspace"       "N"   #'+workspace/new-named
 :desc "Load workspace from file"  "l"   #'+workspace/load
 :desc "Save workspace to file"    "s"   #'+workspace/save
 :desc "Delete session"            "x"   #'+workspace/kill-session
 :desc "Delete this workspace"     "d"   #'+workspace/delete
 :desc "Rename workspace"          "r"   #'+workspace/rename
 :desc "Restore last session"      "R"   #'+workspace/restore-last-session
 :desc "Next workspace"            ">"   #'+workspace/switch-right ;; Modified
 :desc "Previous workspace"        "<"   #'+workspace/switch-left ;; Modified
 :desc "Switch to 1st workspace"   "1"   #'+workspace/switch-to-0
 :desc "Switch to 2nd workspace"   "2"   #'+workspace/switch-to-1
 :desc "Switch to 3rd workspace"   "3"   #'+workspace/switch-to-2
 :desc "Switch to 4th workspace"   "4"   #'+workspace/switch-to-3
 :desc "Switch to 5th workspace"   "5"   #'+workspace/switch-to-4
 :desc "Switch to 6th workspace"   "6"   #'+workspace/switch-to-5
 :desc "Switch to 7th workspace"   "7"   #'+workspace/switch-to-6
 :desc "Switch to 8th workspace"   "8"   #'+workspace/switch-to-7
 :desc "Switch to 9th workspace"   "9"   #'+workspace/switch-to-8
 :desc "Switch to final workspace" "0"   #'+workspace/switch-to-final))
```

7.3.2 Info colors

Better colors for manual pages.

```
(package! info-colors :pin "47ee73cc19b1049eef32c9f3e264ea7ef2aaf8a5")

(use-package! info-colors
  :commands (info-colors-fontify-node))

(add-hook 'Info-selection-hook 'info-colors-fontify-node)
```

7.3.3 The Silver Searcher

An Emacs front-end to *The Silver Searcher*, first we need to install `ag` using `brew install the_silver_searcher`.

```
(package! ag :pin "ed7e32064f92f1315cecbfc43f120bbc7508672c")

(use-package! ag
  :when AG-P
  :commands (ag
             ag-files
             ag-regexp
             ag-project
             ag-project-files
             ag-project-regexp))
```

7.3.4 Page break lines

A feature that displays ugly form feed characters as tidy horizontal rules. Inspired by M-EMACS.

```
(package! page-break-lines :pin "79eca86e0634ac68af862e15c8a236c37f446dcd")

(use-package! page-break-lines
  :diminish
  :init (global-page-break-lines-mode))
```

7.3.5 PDF tools

The `pdf-tools` package supports dark mode (midnight), I use Emacs often to write and read PDF documents, so let's make it dark by default, this can be toggled using the `m z`.

```
(after! pdf-tools
  ;; Auto install
  (pdf-tools-install-noverify)

  (setq-default pdf-view-image-relief 2
                pdf-view-display-size 'fit-page)

  (add-hook! 'pdf-view-mode-hook
    (when (memq doom-theme '(modus-vivendi doom-one doom-dark+ doom-vibrant))
      ;; TODO: find a more generic way to detect if we are in a dark theme
      (pdf-view-midnight-minor-mode 1)))

  ;; Color the background, so we can see the PDF page borders
```

```
  ;; https://protesilaos.com/emacs/modus-themes#h:ff69dfe1-29c0-447a-915c-b5ff7c5509c
  (defun +pdf-tools-backdrop ()
    (face-remap-add-relative
     'default
     `(:background ,(if (memq doom-theme '(modus-vivendi modus-operandi))
                        (modus-themes-color 'bg-alt)
                      (doom-color 'bg-alt)))))

  (add-hook 'pdf-tools-enabled-hook #'+pdf-tools-backdrop))

(after! pdf-links
  ;; Tweak for Modus and `pdf-links'
  (when (memq doom-theme '(modus-vivendi modus-operandi))
    ;; https://protesilaos.com/emacs/modus-themes#h:2659d13e-b1a5-416c-9a89-7c3ce3a76
    (let ((spec (apply #'append
                       (mapcar
                        (lambda (name)
                          (list name
                                (face-attribute 'pdf-links-read-link
                                                name nil 'default)))
                        '(:family :width :weight :slant)))))
      (setq pdf-links-read-link-convert-commands
            `("-density"    "96"
              "-family"     ,(plist-get spec :family)
              "-stretch"    ,(let* ((width (plist-get spec :width))
                                    (name (symbol-name width)))
                               (replace-regexp-in-string "-" ""
                                                         (capitalize name)))
              "-weight"     ,(pcase (plist-get spec :weight)
                               ('ultra-light "Thin")
                               ('extra-light "ExtraLight")
                               ('light       "Light")
                               ('semi-bold   "SemiBold")
                               ('bold        "Bold")
                               ('extra-bold  "ExtraBold")
                               ('ultra-bold  "Black")
                               (_weight      "Normal"))
              "-style"      ,(pcase (plist-get spec :slant)
                               ('italic  "Italic")
                               ('oblique "Oblique")
```

```
                                (_slant   "Normal"))
                "-pointsize"  "%P"
                "-undercolor" "%f"
                "-fill"       "%b"
                "-draw"       "text %X,%Y '%c'")))))
```

7.4 Natural languages

7.4.1 Spell-Fu

Install the `aspell` back-end and the dictionaries to use with `spell-fu`

Now, `spell-fu` supports multiple languages! Let's add English, French and Arabic. So I can "mélanger les langues sans avoir de problèmes!".

```
(after! spell-fu
  (defun +spell-fu-register-dictionary (lang)
    "Add `LANG` to spell-fu multi-dict, with a personal dictionary."
    ;; Add the dictionary
    (spell-fu-dictionary-add (spell-fu-get-ispell-dictionary lang))
    (let ((personal-dict-file (expand-file-name (format "aspell.%s.pws" lang) doom-us
      ;; Create an empty personal dictionary if it doesn't exists
      (unless (file-exists-p personal-dict-file) (write-region "" nil personal-dict-f
      ;; Add the personal dictionary
      (spell-fu-dictionary-add (spell-fu-get-personal-dictionary (format "%s-personal

  (add-hook 'spell-fu-mode-hook
            (lambda ()
              (+spell-fu-register-dictionary +my/lang-main)
              (+spell-fu-register-dictionary +my/lang-secondary))))
```

7.4.2 Proselint

A good and funny linter for English prose!, install via `pip install proselint`.

```
(after! flycheck
  (flycheck-define-checker proselint
    "A linter for prose."
    :command ("proselint" source-inplace)
    :error-patterns
    ((warning line-start (file-name) ":" line ":" column ": "
              (id (one-or-more (not (any " "))))
```

```
            (message) line-end))
    :modes (text-mode markdown-mode gfm-mode org-mode))

  ;; TODO: Can be enabled automatically for English documents using `guess-language'
  (defun +flycheck-proselint-toggle ()
    "Toggle Proselint checker for the current buffer."
    (interactive)
    (if (and (fboundp 'guess-language-buffer) (string= "en" (guess-language-buffer)))
        (if (memq 'proselint flycheck-checkers)
            (setq-local flycheck-checkers (delete 'proselint flycheck-checkers))
          (setq-local flycheck-checkers (append flycheck-checkers '(proselint))))
      (message "Proselint understands only English!"))))
```

7.4.3 Grammarly

Use either eglot-grammarly or lsp-grammarly.

```
(package! grammarly
  :recipe (:host github
           :repo "emacs-grammarly/grammarly")
  :pin "e47b370faace9ca081db0b87ae3bcfd73212c56d")
```

```
(use-package! grammarly
  :config
  (grammarly-load-from-authinfo))
```

Eglot

```
(package! eglot-grammarly
  :disable (not (modulep! :tools lsp +eglot))
  :recipe (:host github
           :repo "emacs-grammarly/eglot-grammarly")
  :pin "3313f38ed7d23947992e19f1e464c6d544124144")
```

```
(use-package! eglot-grammarly
  :commands (+lsp-grammarly-load)
  :init
  (defun +lsp-grammarly-load ()
    "Load Grammarly LSP server for Eglot."
    (interactive)
    (require 'eglot-grammarly)
    (call-interactively #'eglot)))
```

LSP Mode

```
(package! lsp-grammarly
  :disable (or (not (modulep! :tools lsp)) (modulep! :tools lsp +eglot))
  :recipe (:host github
           :repo "emacs-grammarly/lsp-grammarly")
  :pin "eab5292037478c32e7d658fb5cba8b8fb6d72a7c")

(use-package! lsp-grammarly
  :commands (+lsp-grammarly-load +lsp-grammarly-toggle)
  :init
  (defun +lsp-grammarly-load ()
    "Load Grammarly LSP server for LSP Mode."
    (interactive)
    (require 'lsp-grammarly)
    (lsp-deferred)) ;; or (lsp)

  (defun +lsp-grammarly-enabled-p ()
    (not (member 'grammarly-ls lsp-disabled-clients)))

  (defun +lsp-grammarly-enable ()
    "Enable Grammarly LSP."
    (interactive)
    (when (not (+lsp-grammarly-enabled-p))
      (setq lsp-disabled-clients (remove 'grammarly-ls lsp-disabled-clients))
      (message "Enabled grammarly-ls"))
    (+lsp-grammarly-load))

  (defun +lsp-grammarly-disable ()
    "Disable Grammarly LSP."
    (interactive)
    (when (+lsp-grammarly-enabled-p)
      (add-to-list 'lsp-disabled-clients 'grammarly-ls)
      (lsp-disconnect)
      (message "Disabled grammarly-ls")))

  (defun +lsp-grammarly-toggle ()
    "Enable/disable Grammarly LSP."
    (interactive)
    (if (+lsp-grammarly-enabled-p)
```

```
        (+lsp-grammarly-disable)
      (+lsp-grammarly-enable)))

  (after! lsp-mode
    ;; Disable by default
    (add-to-list 'lsp-disabled-clients 'grammarly-ls))

  :config
  (set-lsp-priority! 'grammarly-ls 1))
```

7.4.4 Grammalecte

```
(package! flycheck-grammalecte
  :recipe (:host github
           :repo "milouse/flycheck-grammalecte")
  :pin "314de13247710410f11d060a214ac4f400c02a71")

(use-package! flycheck-grammalecte
  :when nil ;; BUG: Disabled, there is a Python error
  :commands (flycheck-grammalecte-correct-error-at-point
             grammalecte-conjugate-verb
             grammalecte-define
             grammalecte-define-at-point
             grammalecte-find-synonyms
             grammalecte-find-synonyms-at-point)
  :init
  (setq grammalecte-settings-file (expand-file-name "grammalecte/grammalecte-cache.el
        grammalecte-python-package-directory (expand-file-name "grammalecte/grammalec

  (setq flycheck-grammalecte-report-spellcheck t
        flycheck-grammalecte-report-grammar t
        flycheck-grammalecte-report-apos nil
        flycheck-grammalecte-report-esp nil
        flycheck-grammalecte-report-nbsp nil
        flycheck-grammalecte-filters
        '("(?m)^# ?-*-.+$"
          ;; Ignore LaTeX equations (inline and block)
          "\\$.*?\\$"
          "(?s)\\\\begin{\\(?1:\\(?:equation.\\|align.\\)\\)}.*?\\\\end{\\1}"))
```

```
  (map! :leader :prefix ("l" . "custom")
        (:prefix ("g" . "grammalecte")
         :desc "Correct error at point"     "p" #'flycheck-grammalecte-correct-error-
         :desc "Conjugate a verb"           "V" #'grammalecte-conjugate-verb
         :desc "Define a word"              "W" #'grammalecte-define
         :desc "Conjugate a verb at point"  "w" #'grammalecte-define-at-point
         :desc "Find synonyms"              "S" #'grammalecte-find-synonyms
         :desc "Find synonyms at point"     "s" #'grammalecte-find-synonyms-at-point)

  :config
  (grammalecte-download-grammalecte)
  (flycheck-grammalecte-setup))
```

7.4.5 LTeX/LanguageTool

Originally, LTeX LS stands for *LaTeX Language Server*, it acts as a Language Server for LaTeX, but not only. It can check the grammar and the spelling of several markup languages such as BibTeX, ConTeXt, LaTeX, Markdown, Org, reStructuredText… and others. Alongside, it provides interfacing with LanguageTool to implement natural language checking.

```
(after! lsp-ltex
  (setq lsp-ltex-language "auto"
        lsp-ltex-mother-tongue +my/lang-mother-tongue
        flycheck-checker-error-threshold 1000)

  (advice-add
   '+lsp-ltex-setup :after
   (lambda ()
     (setq-local lsp-idle-delay 5.0
                 lsp-progress-function #'lsp-on-progress-legacy
                 lsp-progress-spinner-type 'half-circle
                 lsp-ui-sideline-show-code-actions nil
                 lsp-ui-sideline-show-diagnostics nil
                 lsp-ui-sideline-enable nil)))

  ;; FIXME
  (defun +lsp-ltex-check-document ()
    (interactive)
    (when-let ((file (buffer-file-name)))
```

```
    (let* ((uri (lsp--path-to-uri file))
           (beg (region-beginning))
           (end (region-end))
           (req (if (region-active-p)
                    `(:uri ,uri
                      :range ,(lsp--region-to-range beg end))
                  `(:uri ,uri))))
      (lsp-send-execute-command "_ltex.checkDocument" req)))))
```

7.4.6 Go Translate (Google, Bing and DeepL)

```
(package! go-translate
  :recipe (:host github
           :repo "lorniu/go-translate")
  :pin "8bbcbce42a7139f079df3e9b9bda0def2cbb690f")

(use-package! go-translate
  :commands (gts-do-translate
             +gts-yank-translated-region
             +gts-translate-with)
  :init
  ;; Your languages pairs
  (setq gts-translate-list (list (list +my/lang-main +my/lang-secondary)
                                 (list +my/lang-main +my/lang-mother-tongue)
                                 (list +my/lang-secondary +my/lang-mother-tongue)
                                 (list +my/lang-secondary +my/lang-main)))

  (map! :localleader
        :map (org-mode-map markdown-mode-map latex-mode-map text-mode-map)
        :desc "Yank translated region" "R" #'+gts-yank-translated-region)

  (map! :leader :prefix "l"
        (:prefix ("G" . "go-translate")
         :desc "Bing"                   "b" (lambda () (interactive) (+gts-translate-
         :desc "DeepL"                  "d" (lambda () (interactive) (+gts-translate-
         :desc "Google"                 "g" (lambda () (interactive) (+gts-translate-
         :desc "Yank translated region" "R" #'+gts-yank-translated-region
         :desc "gts-do-translate"       "t" #'gts-do-translate))

  :config
```

```
;; Config the default translator, which will be used by the command `gts-do-transla
(setq gts-default-translator
      (gts-translator
       ;; Used to pick source text, from, to. choose one.
       :picker (gts-prompt-picker)
       ;; One or more engines, provide a parser to give different output.
       :engines (gts-google-engine :parser (gts-google-summary-parser))
       ;; Render, only one, used to consumer the output result.
       :render (gts-buffer-render)))

;; Custom texter which remove newlines in the same paragraph
(defclass +gts-translate-paragraph (gts-texter) ())

(cl-defmethod gts-text ((_ +gts-translate-paragraph))
  (when (use-region-p)
    (let ((text (buffer-substring-no-properties (region-beginning) (region-end))))
      (with-temp-buffer
        (insert text)
        (goto-char (point-min))
        (let ((case-fold-search nil))
          (while (re-search-forward "\n[^\n]" nil t)
            (replace-region-contents
             (- (point) 2) (- (point) 1)
             (lambda (&optional a b) " ")))
          (buffer-string))))))

;; Custom picker to use the paragraph texter
(defclass +gts-paragraph-picker (gts-picker)
  ((texter :initarg :texter :initform (+gts-translate-paragraph))))

(cl-defmethod gts-pick ((o +gts-paragraph-picker))
  (let ((text (gts-text (oref o texter))))
    (when (or (null text) (zerop (length text)))
      (user-error "Make sure there is any word at point, or selection exists"))
    (let ((path (gts-path o text)))
      (setq gts-picker-current-path path)
      (cl-values text path))))

(defun +gts-yank-translated-region ()
  (interactive)
```

```
    (gts-translate
     (gts-translator
      :picker (+gts-paragraph-picker)
      :engines (gts-google-engine)
      :render (gts-kill-ring-render))))

  (defun +gts-translate-with (&optional engine)
    (interactive)
    (gts-translate
     (gts-translator
      :picker (+gts-paragraph-picker)
      :engines
      (cond ((eq engine 'deepl)
             (gts-deepl-engine
              :auth-key ;; Get API key from ~/.authinfo.gpg (machine api-free.deepl.c
              (funcall
               (plist-get (car (auth-source-search :host "api-free.deepl.com" :max 1)
                          :secret))
              :pro nil))
            ((eq engine 'bing) (gts-bing-engine))
            (t (gts-google-engine)))
      :render (gts-buffer-render)))))
```

7.4.7 Offline dictionaries

Needs `sdcv` to be installed, needs also *StarDict* dictionaries, you can download some from here and here for french.

```
(package! lexic
  :recipe (:host github
           :repo "tecosaur/lexic")
  :pin "f9b3de4d9c2dd1ce5022383e1a504b87bf7d1b09")

(use-package! lexic
  :commands (lexic-search lexic-list-dictionary)
  :config
  (map! :map lexic-mode-map
        :n "q" #'lexic-return-from-lexic
        :nv "RET" #'lexic-search-word-at-point
        :n "a" #'outline-show-all
        :n "h" (cmd! (outline-hide-sublevels 3))
```

```
        :n "o" #'lexic-toggle-entry
        :n "n" #'lexic-next-entry
        :n "N" (cmd! (lexic-next-entry t))
        :n "p" #'lexic-previous-entry
        :n "P" (cmd! (lexic-previous-entry t))
        :n "E" (cmd! (lexic-return-from-lexic) ; expand
                     (switch-to-buffer (lexic-get-buffer)))
        :n "M" (cmd! (lexic-return-from-lexic) ; minimise
                     (lexic-goto-lexic))
        :n "C-p" #'lexic-search-history-backwards
        :n "C-n" #'lexic-search-history-forwards
        :n "/" (cmd! (call-interactively #'lexic-search))))
```

8 Completion & IDE

8.1 UI Package

8.1.1 Which key

Make `which-key` popup faster.

```
(setq which-key-idle-delay 0.5 ;; Default is 1.0
      which-key-idle-secondary-delay 0.05) ;; Default is nil
```

- Tôi sử ăn cắp config này từ "tecosaur's config", nó giúp replace các evil `prefix` với unicode symbok, giúp cho which-key xịn xò hơn.

```
(setq which-key-allow-multiple-replacements t)

(after! which-key
  (pushnew! which-key-replacement-alist
            '(""       . "\\`+?evil[-:]?\\(?:a-\\)?\\(.*\\)") . (nil . " ·\\1"))
            '(("\\`g s" . "\\`evilem--?motion-\\(.*\\)")       . (nil . " ·\\1"))))
```

8.1.2 Company

Disabled for some mode

- Tắt chức năng Company trên một vài mode không hữu dụng.

```
(setq company-global-modes
      '(not erc-mode
```

```
            circe-mode
            message-mode
            help-mode
            gud-mode
            vterm-mode
            org-mode))
```

Tweak company-box

```
(after! company-box
  (defun +company-box--reload-icons-h ()
    (setq company-box-icons-all-the-icons
          (let ((all-the-icons-scale-factor 0.8))
            `((Unknown       . ,(all-the-icons-faicon   "code"                 :face
              (Text          . ,(all-the-icons-material "text_fields"          :face
              (Method        . ,(all-the-icons-faicon   "cube"                 :face
              (Function      . ,(all-the-icons-faicon   "cube"                 :face
              (Constructor   . ,(all-the-icons-faicon   "cube"                 :face
              (Field         . ,(all-the-icons-faicon   "tag"                  :face
              (Variable      . ,(all-the-icons-material "adjust"               :face
              (Class         . ,(all-the-icons-material "class"                :face
              (Interface     . ,(all-the-icons-material "tune"                 :face
              (Module        . ,(all-the-icons-faicon   "cubes"                :face
              (Property      . ,(all-the-icons-faicon   "wrench"               :face
              (Unit          . ,(all-the-icons-material "straighten"           :face
              (Value         . ,(all-the-icons-material "filter_1"             :face
              (Enum          . ,(all-the-icons-material "plus_one"             :face
              (Keyword       . ,(all-the-icons-material "filter_center_focus"  :face
              (Snippet       . ,(all-the-icons-faicon   "expand"               :face
              (Color         . ,(all-the-icons-material "colorize"             :face
              (File          . ,(all-the-icons-material "insert_drive_file"    :face
              (Reference     . ,(all-the-icons-material "collections_bookmark" :face
              (Folder        . ,(all-the-icons-material "folder"               :face
              (EnumMember    . ,(all-the-icons-material "people"               :face
              (Constant      . ,(all-the-icons-material "pause_circle_filled"  :face
              (Struct        . ,(all-the-icons-material "list"                 :face
              (Event         . ,(all-the-icons-material "event"                :face
              (Operator      . ,(all-the-icons-material "control_point"        :face
              (TypeParameter . ,(all-the-icons-material "class"                :face
              (Template      . ,(all-the-icons-material "settings_ethernet"    :face
```

```
            (ElispFunction . ,(all-the-icons-faicon   "cube"                 :face
            (ElispVariable . ,(all-the-icons-material "adjust"               :face
            (ElispFeature  . ,(all-the-icons-material "stars"                :face
            (ElispFace     . ,(all-the-icons-material "format_paint"         :face

  (when (daemonp)
    ;; Replace Doom defined icons with mine
    (when (memq #'+company-box--load-all-the-icons server-after-make-frame-hook)
      (remove-hook 'server-after-make-frame-hook #'+company-box--load-all-the-icons))
    (add-hook 'server-after-make-frame-hook #'+company-box--reload-icons-h))

  ;; Reload icons even if not in Daemon mode
  (+company-box--reload-icons-h))
```

8.1.3 Treemacs

```
(after! treemacs
  (require 'dired)

  ;; My custom stuff (from tecosaur's config)
  (setq +treemacs-file-ignore-extensions
        '(;; LaTeX
          "aux" "ptc" "fdb_latexmk" "fls" "synctex.gz" "toc"
          ;; LaTeX - bibliography
          "bbl"
          ;; LaTeX - glossary
          "glg" "glo" "gls" "glsdefs" "ist" "acn" "acr" "alg"
          ;; LaTeX - pgfplots
          "mw"
          ;; LaTeX - pdfx
          "pdfa.xmpi"
          ;; Python
          "pyc"))

  (setq +treemacs-file-ignore-globs
        '(;; LaTeX
          "*/_minted-*"
          ;; AucTeX
          "*/.auctex-auto"
          "*/_region_.log"
```

```
        "*/_region_.tex"
        ;; Python
        "*/__pycache__"))

;; Reload treemacs theme
(setq doom-themes-treemacs-enable-variable-pitch nil
      doom-themes-treemacs-theme "doom-colors")
(doom-themes-treemacs-config)

(setq treemacs-show-hidden-files nil
      treemacs-hide-dot-git-directory t
      treemacs-width 30)

(defvar +treemacs-file-ignore-extensions '()
  "File extension which `treemacs-ignore-filter' will ensure are ignored")

(defvar +treemacs-file-ignore-globs '()
  "Globs which will are transformed to `+treemacs-file-ignore-regexps' which `+tree

(defvar +treemacs-file-ignore-regexps '()
  "RegExps to be tested to ignore files, generated from `+treeemacs-file-ignore-glo

(defun +treemacs-file-ignore-generate-regexps ()
  "Generate `+treemacs-file-ignore-regexps' from `+treemacs-file-ignore-globs'"
  (setq +treemacs-file-ignore-regexps (mapcar 'dired-glob-regexp +treemacs-file-ign

(unless (equal +treemacs-file-ignore-globs '())
  (+treemacs-file-ignore-generate-regexps))

(defun +treemacs-ignore-filter (file full-path)
  "Ignore files specified by `+treemacs-file-ignore-extensions', and `+treemacs-fil
  (or (member (file-name-extension file) +treemacs-file-ignore-extensions)
      (let ((ignore-file nil))
        (dolist (regexp +treemacs-file-ignore-regexps ignore-file)
          (setq ignore-file (or ignore-file (if (string-match-p regexp full-path) t

(add-to-list 'treemacs-ignored-file-predicates #'+treemacs-ignore-filter))
```

8.1.4 Projectile

- Vũ sử dụng Projectile để quản lý project, Doom cũng hỗ trợ quản lý bằng doom-project nhưng vũ sử dụng projectile từ vanilla qua nên cảm thấy nó sử dụng tốt hơn.

```
;; Run `M-x projectile-discover-projects-in-search-path' to reload paths from this va
(setq projectile-project-search-path
      '("~/Dropbox/PhD/papers"
        "~/Dropbox/PhD/workspace"
        "~/Dropbox/PhD/workspace-no"
        "~/Dropbox/PhD/workspace-no/ez-wheel/swd-starter-kit-repo"
        ("~/myProjects/" . 2))) ;; ("dir" . depth)

(setq projectile-ignored-projects
      '("/tmp"
        "~/"
        "~/.cache"
        "~/.doom.d"
        "~/.emacs.d/.local/straight/repos/"))

(setq +projectile-ignored-roots
      '("~/.cache"
        ;; No need for this one, as `doom-project-ignored-p' checks for files in `doo
        "~/.emacs.d/.local/straight/"))

(defun +projectile-ignored-project-function (filepath)
  "Return t if FILEPATH is within any of `+projectile-ignored-roots'"
  (require 'cl-lib)
  (or (doom-project-ignored-p filepath) ;; Used by default by doom with `projectile-i
      (cl-some (lambda (root) (file-in-directory-p (expand-file-name filepath) (expan
          +projectile-ignored-roots)))

(setq projectile-ignored-project-function #'+projectile-ignored-project-function)
```

8.1.5 Tramp

Let's try to make tramp handle prompts better

```
(after! tramp
  (setenv "SHELL" "/bin/bash")
```

```
  (setq tramp-shell-prompt-pattern "\\(?:^\\|
\\)[^]#$%>\n]*#?[]#$%> ] *\\(\\[[0-9;]*[a-zA-Z] *\\)*")) ;; default +
```

8.1.6 Eros-eval

This makes the result of evals slightly prettier.

```
(setq eros-eval-result-prefix "  ")
```

8.1.7 `dir-locals.el`

Reload `dir-locals.el` variables after modification. Taken from this answer.

```
(defun +dir-locals-reload-for-current-buffer ()
  "reload dir locals for the current buffer"
  (interactive)
  (let ((enable-local-variables :all))
    (hack-dir-local-variables-non-file-buffer)))

(defun +dir-locals-reload-for-all-buffers-in-this-directory ()
  "For every buffer with the same `default-directory` as the
current buffer's, reload dir-locals."
  (interactive)
  (let ((dir default-directory))
    (dolist (buffer (buffer-list))
      (with-current-buffer buffer
        (when (equal default-directory dir)
          (+dir-locals-reload-for-current-buffer))))))

(defun +dir-locals-enable-autoreload ()
  (when (and (buffer-file-name)
             (equal dir-locals-file (file-name-nondirectory (buffer-file-name))))
    (message "Dir-locals will be reloaded after saving.")
    (add-hook 'after-save-hook '+dir-locals-reload-for-all-buffers-in-this-directory

(add-hook! '(emacs-lisp-mode-hook lisp-data-mode-hook) #'+dir-locals-enable-autoreloa
```

8.1.8 Erefactor

```
(package! erefactor
```

```
  :recipe (:host github
           :repo "mhayashi1120/Emacs-erefactor")
  :pin "bfe27a1b8c7cac0fe054e76113e941efa3775fe8")

(use-package! erefactor
  :defer t)
```

8.1.9 Lorem ipsum

```
(package! emacs-lorem-ipsum
  :recipe (:host github
           :repo "jschaf/emacs-lorem-ipsum")
  :pin "da75c155da327c7a7aedb80f5cfe409984787049")

(use-package! lorem-ipsum
  :commands (lorem-ipsum-insert-sentences
             lorem-ipsum-insert-paragraphs
             lorem-ipsum-insert-list))
```

8.1.10 Coverage test

```
(package! cov :pin "cd3e1995c596cc227124db9537792d8329ffb696")
```

8.2 Languages

8.2.1 Language Server Protocol

Eglot Eglot uses `project.el` to detect the project root. This is a workaround to make it work with `projectile`:

```
(after! eglot
  ;; A hack to make it works with projectile
  (defun projectile-project-find-function (dir)
    (let* ((root (projectile-project-root dir)))
      (and root (cons 'transient root))))

  (with-eval-after-load 'project
    (add-to-list 'project-find-functions 'projectile-project-find-function))

  ;; Use clangd with some options
  (set-eglot-client! 'c++-mode '("clangd" "-j=3" "--clang-tidy")))
```

LSP mode

Unpin package

```
(unpin! lsp-mode)
```

Tweaks

1. Performance Use `plist` instead of hash table, LSP mode needs to be reinstalled after setting this environment variable (see).

   ```
   (after! lsp-mode
     (setq lsp-idle-delay 1.0
           lsp-log-io nil
           gc-cons-threshold (* 1024 1024 100))) ;; 100MiB
   ```

2. Features & UI LSP mode provides a set of configurable UI stuff. By default, Doom Emacs disables some UI components; however, I like to enable some less intrusive, more useful UI stuff.

   ```
   (after! lsp-mode
     (setq lsp-lens-enable t
           lsp-semantic-tokens-enable t ;; hide unreachable ifdefs
           lsp-enable-symbol-highlighting t
           lsp-headerline-breadcrumb-enable nil
           ;; LSP UI related tweaks
           lsp-ui-sideline-enable nil
           lsp-ui-sideline-show-hover nil
           lsp-ui-sideline-show-symbol nil
           lsp-ui-sideline-show-diagnostics nil
           lsp-ui-sideline-show-code-actions nil))
   ```

LSP mode with `clangd`

```
(after! lsp-clangd
  (setq lsp-clients-clangd-args
        '("-j=4"
          "--background-index"
          "--clang-tidy"
          "--completion-style=detailed"
          "--header-insertion=never"
          "--header-insertion-decorators=0"))
  (set-lsp-priority! 'clangd 1))
```

LSP mode with `ccls`

```
;; NOTE: Not tangled, using the default ccls
(after! ccls
  (setq ccls-initialization-options
        '(:index (:comments 2
                  :trackDependency 1
                  :threads 4)
          :completion (:detailedLabel t)))
  (set-lsp-priority! 'ccls 2)) ; optional as ccls is the default in Doom
```

Enable `lsp` over `tramp`

1. Python

```
(after! tramp
  (when (require 'lsp-mode nil t)
    ;; (require 'lsp-pyright)

    (setq lsp-enable-snippet nil
          lsp-log-io nil
          ;; To bypass the "lsp--document-highlight fails if
          ;; textDocument/documentHighlight is not supported" error
          lsp-enable-symbol-highlighting nil)

    (lsp-register-client
     (make-lsp-client
      :new-connection (lsp-tramp-connection "pyls")
      :major-modes '(python-mode)
      :remote? t
      :server-id 'pyls-remote))))
```

2. C/C++ with `ccls`

```
;; NOTE: WIP: Not tangled
(after! tramp
  (when (require 'lsp-mode nil t)
    (require 'ccls)

    (setq lsp-enable-snippet nil
```

```
        lsp-log-io nil
        lsp-enable-symbol-highlighting t)

    (lsp-register-client
     (make-lsp-client
      :new-connection
      (lsp-tramp-connection
       (lambda ()
         (cons ccls-executable ; executable name on remote machine 'ccls'
               ccls-args)))
      :major-modes '(c-mode c++-mode objc-mode cuda-mode)
      :remote? t
      :server-id 'ccls-remote)))

  (add-to-list 'tramp-remote-path 'tramp-own-remote-path))
```

3. C/C++ with `clangd`

```
(after! tramp
  (when (require 'lsp-mode nil t)

    (setq lsp-enable-snippet nil
          lsp-log-io nil
          ;; To bypass the "lsp--document-highlight fails if
          ;; textDocument/documentHighlight is not supported" error
          lsp-enable-symbol-highlighting nil)

    (lsp-register-client
     (make-lsp-client
      :new-connection
      (lsp-tramp-connection
       (lambda ()
         (cons "clangd-12" ; executable name on remote machine 'ccls'
               lsp-clients-clangd-args)))
      :major-modes '(c-mode c++-mode objc-mode cuda-mode)
      :remote? t
      :server-id 'clangd-remote))))
```

VHDL By default, LSP uses the proprietary VHDL-Tool to provide LSP features; however, there is free and open source alternatives: `ghdl-ls`

and `rust_hdl`. I have some issues running `ghdl-ls` installed form `pip` through the `pyghdl` package, so let's use `rust_hdl` instead.

```
(use-package! vhdl-mode
  :when (and (modulep! :tools lsp) (not (modulep! :tools lsp +eglot)))
  :hook (vhdl-mode . #'+lsp-vhdl-ls-load)
  :init
  (defun +lsp-vhdl-ls-load ()
    (interactive)
    (lsp t)
    (flycheck-mode t))

  :config
  ;; Required unless vhdl_ls is on the $PATH
  (setq lsp-vhdl-server-path "~/Projects/foss/repos/rust_hdl/target/release/vhdl_ls"
        lsp-vhdl-server 'vhdl-ls
        lsp-vhdl--params nil)
  (require 'lsp-vhdl))
```

SonarLint

```
(package! lsp-sonarlint
  :disable t :pin "3313f38ed7d23947992e19f1e464c6d544124144")

(use-package! lsp-sonarlint)
```

8.2.2 C++

Cppcheck Check for everything!

```
(after! flycheck
  (setq flycheck-cppcheck-checks '("information"
                                   "missingInclude"
                                   "performance"
                                   "portability"
                                   "style"
                                   "unusedFunction"
                                   "warning"))) ;; Actually, we can use "all"
```

Project CMake

- Một package mới tương tác rất tối với cmake project, nó là sự kết hợp của project, eglot, cmake và clangd.

```
(package! project-cmake
  :disable t ; (not (modulep! :tools lsp +eglot)) ; Enable only if (lsp +eglot) is us
  :pin "3313f38ed7d23947992e19f1e464c6d544124144"
  :recipe (:host github
           :repo "juanjosegarciaripoll/project-cmake"))
```

```
(use-package! project-cmake
    :config
    (require 'eglot)
    (project-cmake-scan-kits)
    (project-cmake-eglot-integration))
```

Clang-format

```
(package! clang-format :pin "e48ff8ae18dc7ab6118c1f6752deb48cb1fc83ac")
```

```
(use-package! clang-format
  :when CLANG-FORMAT-P
  :commands (clang-format-region))
```

Auto-include C++ headers

```
(package! cpp-auto-include
  :recipe (:host github
           :repo "emacsorphanage/cpp-auto-include")
  :pin "0ce829f27d466c083e78b9fe210dcfa61fb417f4")
```

```
(use-package! cpp-auto-include
  :commands cpp-auto-include)
```

C/C++ preprocessor conditions

- Trong trường hợp LSP không khả dụng, chúng ta sử dụng ifdef.

```
(unless (modulep! :lang cc +lsp) ;; Disable if LSP for C/C++ is enabled
  (use-package! hideif
    :hook (c-mode . hide-ifdef-mode)
```

```
    :hook (c++-mode . hide-ifdef-mode)
    :init
    (setq hide-ifdef-shadow t
          hide-ifdef-initially t)))
```

8.2.3 Solidity

Company

```
(use-package! company-solidity
  :after (company))


(use-package! solidity-mode
  :config
  (setq format-all-mode nil))
(setq-hook! 'solidity-mode-hook +format-all-mode nil)

(add-hook 'solidity-mode-hook
          (lambda ()
            (set (make-local-variable 'company-backends)
                 (append '((company-solidity company-capf company-dabbrev-code))
                         company-backends))))
```

flycheck

```
(use-package! solidity-flycheck
  :config
  (setq solidity-solc-path "~/.nvm/version/node/v16.17.0/lib/node_modules/solc/solc")
  (setq solidity-solium-path "~/.nvm/version/node/v16.17.0/bin/solium")
  (setq solidity-flycheck-solc-checker-active t)
  (setq solidity-flycheck-solium-checker-active t)
  (setq flycheck-solidity-solc-addstd-contracts t)
  (setq flycheck-solidity-solium-soliumrcfile "~/.soliumrc.json")
)
```

8.3 Debugging

8.3.1 DAP

when the program hits a break point, and automatically delete the session and close Hydra when DAP is terminated.

```
(unpin! dap-mode)
```

```
(after! dap-mode
  ;; Set latest versions
  (setq dap-cpptools-extension-version "1.11.5")
  (require 'dap-cpptools)

  (setq dap-codelldb-extension-version "1.7.4")
  (require 'dap-codelldb)

  (setq dap-gdb-lldb-extension-version "0.26.0")
  (require 'dap-gdb-lldb)

  ;; More minimal UI
  (setq dap-auto-configure-features '(breakpoints locals expressions tooltip)
        dap-auto-show-output nil ;; Hide the annoying server output
        lsp-enable-dap-auto-configure t)

  ;; Automatically trigger dap-hydra when a program hits a breakpoint.
  (add-hook 'dap-stopped-hook (lambda (arg) (call-interactively #'dap-hydra)))

  ;; Automatically delete session and close dap-hydra when DAP is terminated.
  (add-hook 'dap-terminated-hook
            (lambda (arg)
              (call-interactively #'dap-delete-session)
              (dap-hydra/nil)))

  ;; A workaround to correctly show breakpoints
  ;; from: https://github.com/emacs-lsp/dap-mode/issues/374#issuecomment-1140399819
  (add-hook! +dap-running-session-mode
    (set-window-buffer nil (current-buffer))))
```

Doom store Doom Emacs stores session information persistently using the core `store` mechanism. However, relaunching a new session doesn't overwrite the last stored session, to do so, I define a helper function to clear data

stored in the "+debugger" location. (see +debugger--get-last-config function.)

```
(defun +debugger/clear-last-session ()
  "Clear the last stored session"
  (interactive)
  (doom-store-clear "+debugger"))

(map! :leader :prefix ("l" . "custom")
      (:when (modulep! :tools debugger +lsp)
       :prefix ("d" . "debugger")
       :desc "Clear last DAP session" "c" #'+debugger/clear-last-session))
```

8.4 Development Config

8.4.1 multi-iedit

```
 (use-package! maple-iedit
    :commands (maple-iedit-match-all maple-iedit-match-next maple-iedit-match-previou
    :config
    (delete-selection-mode t)
    (setq maple-iedit-ignore-case t)
    (defhydra maple/iedit ()
      ("n" maple-iedit-match-next "next")
      ("t" maple-iedit-skip-and-match-next "skip and next")
      ("T" maple-iedit-skip-and-match-previous "skip and previous")
      ("p" maple-iedit-match-previous "prev"))
    :bind (:map evil-visual-state-map
           ("n" . maple/iedit/body)
           ("C-n" . maple-iedit-match-next)
           ("C-p" . maple-iedit-match-previous)
           ("C-t" . map-iedit-skip-and-match-next)
           ("C-T" . map-iedit-skip-and-match-previous)))
```

8.4.2 engine-mode

```
(package! engine-mode)
```

- Search on browser

```
 (use-package engine-mode
```

```
:config
(engine/set-keymap-prefix (kbd "C-c s"))
(setq browse-url-browser-function 'browse-url-default-macosx-browser
      engine/browser-function 'browse-url-default-macosx-browser
      ;; browse-url-generic-program "google-chrome"
      )
(defengine duckduckgo
  "https://duckduckgo.com/?q=%s"
  :keybinding "d")

(defengine github
  "https://github.com/search?ref=simplesearch&q=%s"
  :keybinding "1")

(defengine gitlab
  "https://gitlab.com/search?search=%s&group_id=&project_id=&snippets=false&repo
  :keybinding "2")

(defengine stack-overflow
  "https://stackoverflow.com/search?q=%s"
  :keybinding "s")

(defengine npm
  "https://www.npmjs.com/search?q=%s"
  :keybinding "n")

(defengine crates
  "https://crates.io/search?q=%s"
  :keybinding "c")

(defengine localhost
  "http://localhost:%s"
  :keybinding "l")

(defengine translate
  "https://translate.google.com/?sl=en&tl=vi&text=%s&op=translate"
  :keybinding "t")

(defengine youtube
  "http://www.youtube.com/results?aq=f&oq=&search_query=%s"
```

```
      :keybinding "y")

    (defengine google
      "http://www.google.com/search?ie=utf-8&oe=utf-8&q=%s"
      :keybinding "g")

    (engine-mode 1))
```

8.4.3 leetcode

```
(after! leetcode
  (setq leetcode-prefer-language "rust"
        leetcode-prefer-sql "mysql"
        leetcode-save-solutions t
        leetcode-directory "~/Dropbox/vugomars/leetcode")
  (set-popup-rule! "^\\*leetcode" :actions '(open-popup-on-side-or-below)))
```

8.4.4 bm

```
(package! bm)

(use-package bm
         :ensure t
         :demand t

         :init
         ;; restore on load (even before you require bm)
         (setq bm-restore-repository-on-load t)


         :config
         ;; Allow cross-buffer 'next'
         (setq bm-cycle-all-buffers t)

         ;; where to store persistant files
         (setq bm-repository-file "~/.emacs.d/bm-repository")

         ;; save bookmarks
         (setq-default bm-buffer-persistence t)

         ;; Loading the repository from file when on start up.
```

```
(add-hook 'after-init-hook 'bm-repository-load)

;; Saving bookmarks
(add-hook 'kill-buffer-hook #'bm-buffer-save)

;; Saving the repository to file when on exit.
;; kill-buffer-hook is not called when Emacs is killed, so we
;; must save all bookmarks first.
(add-hook 'kill-emacs-hook #'(lambda nil
                                  (bm-buffer-save-all)
                                  (bm-repository-save)))

;; The `after-save-hook' is not necessary to use to achieve persistence,
;; but it makes the bookmark data in repository more in sync with the file
;; state.
(add-hook 'after-save-hook #'bm-buffer-save)

;; Restoring bookmarks
(add-hook 'find-file-hooks   #'bm-buffer-restore)
(add-hook 'after-revert-hook #'bm-buffer-restore)

;; The `after-revert-hook' is not necessary to use to achieve persistence,
;; but it makes the bookmark data in repository more in sync with the file
;; state. This hook might cause trouble when using packages
;; that automatically reverts the buffer (like vc after a check-in).
;; This can easily be avoided if the package provides a hook that is
;; called before the buffer is reverted (like `vc-before-checkin-hook').
;; Then new bookmarks can be saved before the buffer is reverted.
;; Make sure bookmarks is saved before check-in (and revert-buffer)
(add-hook 'vc-before-checkin-hook #'bm-buffer-save)


:bind (("C-s-0" . bm-toggle)
       ("C-s-j" . bm-next)
       ("C-s-k" . bm-previous))
)
```

9 Application

9.1 Maxima

The Maxima CAS cames bundled with three Emacs modes: `maxima`, `imaxima` and `emaxima`; installed by default in `"/usr/share/emacs/site-lisp/maxima"`.

9.1.1 Maxima

The emacsmirror/maxima seems more up-to-date, and supports completion via Company, so let's install it from GitHub. Note that, normally, we don't need to specify a recipe; however, installing it directly seems to not install `company-maxima.el` and `poly-maxima.el`.

```
(package! maxima
  :recipe (:host github
           :repo "emacsmirror/maxima"
           :files (:defaults
                   "keywords"
                   "company-maxima.el"
                   "poly-maxima.el"))
  :pin "1334f44725bd80a265de858d652f3fde4ae401fa")

(use-package! maxima
  :when MAXIMA-P
  :commands (maxima-mode maxima-inferior-mode maxima)
  :init
  (require 'straight) ;; to use `straight-build-dir' and `straight-base-dir'
  (setq maxima-font-lock-keywords-directory ;; a workaround to undo the straight work
        (expand-file-name (format "straight/%s/maxima/keywords" straight-build-dir) s

  ;; The `maxima-hook-function' setup `company-maxima'.
  (add-hook 'maxima-mode-hook #'maxima-hook-function)
  (add-hook 'maxima-inferior-mode-hook #'maxima-hook-function)
  (add-to-list 'auto-mode-alist '("\\.ma[cx]\\'" . maxima-mode)))
```

9.1.2 IMaxima

For the `imaxima` (Maxima with image support), the emacsattic/imaxima seems outdated compared to the `imaxima` package of the official Maxima distribution, so let's install `imaxima` from the source code of Maxima, hosted

on Sourceforge git.code.sf.net/p/maxima/code. The package files are stored in the repository's subdirectory `interfaces/emacs/imaxima`.

```
;; Use the `imaxima' package bundled with the official Maxima distribution.
(package! imaxima
  :recipe (:host nil ;; Unsupported host, we will specify the complete repo link
           :repo "https://git.code.sf.net/p/maxima/code"
           :files ("interfaces/emacs/imaxima/*"))
  :pin "519ea34095e749634d3a188733a3ad284b593e12")

(use-package! imaxima
  :when MAXIMA-P
  :commands (imaxima imath-mode)
  :init
  (setq imaxima-use-maxima-mode-flag nil ;; otherwise, it don't render equations with
        imaxima-scale-factor 2.0)

  ;; Hook the `maxima-inferior-mode' to get Company completion.
  (add-hook 'imaxima-startup-hook #'maxima-inferior-mode))
```

10 Programming

10.1 CSV rainbow

Stolen from here.

```
(after! csv-mode
  ;; TODO: Need to fix the case of two commas, example "a,b,,c,d"
  (require 'cl-lib)
  (require 'color)

  (map! :localleader
        :map csv-mode-map
        "R" #'+csv-rainbow)

  (defun +csv-rainbow (&optional separator)
    (interactive (list (when current-prefix-arg (read-char "Separator: "))))
    (font-lock-mode 1)
    (let* ((separator (or separator ?\,))
           (n (count-matches (string separator) (point-at-bol) (point-at-eol)))
```

```
          (colors (cl-loop for i from 0 to 1.0 by (/ 2.0 n)
                           collect (apply #'color-rgb-to-hex
                                          (color-hsl-to-rgb i 0.3 0.5)))))
      (cl-loop for i from 2 to n by 2
               for c in colors
               for r = (format "^\\([^%c\n]+%c\\)\\{%d\\}" separator separator i)
               do (font-lock-add-keywords nil `((,r (1 '(face (:foreground ,c))))))))

;; provide CSV mode setup
;; (add-hook 'csv-mode-hook (lambda () (+csv-rainbow)))
```

10.2 Python IDE

```
(package! elpy :pin "de31d30003c515c25ff7bfd3a361c70c298f78bb")

(use-package! elpy
  :hook ((elpy-mode . flycheck-mode)
         (elpy-mode . (lambda ()
                        (set (make-local-variable 'company-backends)
                             '((elpy-company-backend :with company-yasnippet))))))
  :config
  (elpy-enable))
```

10.3 Scheme

```
(after! geiser
  (setq geiser-default-implementation 'guile
        geiser-chez-binary "chez-scheme")) ;; default is "scheme"
```

10.4 Git & VC

10.4.1 Magit

```
(after! code-review
  (setq code-review-auth-login-marker 'forge))
```

Granular diff-highlights for *all* hunks

```
(after! magit
  ;; Disable if it causes performance issues
  (setq magit-diff-refine-hunk t))
```

Gravatars

```
(after! magit
  ;; Show gravatars
  (setq magit-revision-show-gravatars '("^Author:     " . "^Commit:     ")))
```

WIP Company for commit messages

```
(package! company-conventional-commits
  :recipe `(:local-repo ,(expand-file-name "lisp/company-conventional-commits" doom-u

(use-package! company-conventional-commits
  :after (magit company)
  :config
  (add-hook
   'git-commit-setup-hook
   (lambda ()
     (add-to-list 'company-backends 'company-conventional-commits))))
```

Pretty graph

```
(package! magit-pretty-graph
  :recipe (:host github
           :repo "georgek/magit-pretty-graph")
  :pin "26dc5535a20efe781b172bac73f14a5ebe13efa9")

(use-package! magit-pretty-graph
  :after magit
  :init
  (setq magit-pg-command
        (concat "git --no-pager log"
                " --topo-order --decorate=full"
                " --pretty=format:\"%H%x00%P%x00%an%x00%ar%x00%s%x00%d\""
                " -n 2000")) ;; Increase the default 100 limit

  (map! :localleader
        :map (magit-mode-map)
        :desc "Magit pretty graph" "p" (cmd! (magit-pg-repo (magit-toplevel)))))
```

10.4.2 Repo

This adds Emacs integration of `repo`, The Multiple Git Repository Tool. Make sure the repo tool is installed, if not, `pacman -S repo` on Arch-based distributions, or directly with:

```
REPO_PATH="$HOME/.local/bin/repo"
curl "https://storage.googleapis.com/git-repo-downloads/repo" > "${REPO_PATH}"
chmod a+x "${REPO_PATH}"

(package! repo :pin "e504aa831bfa38ddadce293face28b3c9d9ff9b7")

(use-package! repo
  :when REPO-P
  :commands repo-status)
```

10.4.3 Blamer

Display Git information (author, date, message…) for current line

```
(package! blamer
  :recipe (:host github
           :repo "artawower/blamer.el")
  :pin "99b43779341af0d924bfe2a9103993a6b9e3d3b2")

(use-package! blamer
  :commands (blamer-mode)
  ;; :hook ((prog-mode . blamer-mode))
  :custom
  (blamer-idle-time 0.3)
  (blamer-min-offset 60)
  (blamer-prettify-time-p t)
  (blamer-entire-formatter "    %s")
  (blamer-author-formatter "  %s ")
  (blamer-datetime-formatter "[%s], ")
  (blamer-commit-formatter ""%s"")
  :custom-face
  (blamer-face ((t :foreground "#7a88cf"
                   :background nil
                   :height 125
                   :italic t))))
```

10.5 ESS

View data frames better with

```
(package! ess-view :pin "925cafd876e2cc37bc756bb7fcf3f34534b457e2")
```

10.6 Semgrep

Lightweight static analysis for many languages. Find bug variants with patterns that look like source code.

```
(package! semgrep
  :disable t
  :recipe (:host github
           :repo "Ruin0x11/semgrep.el")
  :pin "3313f38ed7d23947992e19f1e464c6d544124144")
```

10.7 Assembly

Add some packages for better assembly coding.

```
(package! nasm-mode :pin "65ca6546fc395711fac5b3b4299e76c2303d43a8")
(package! haxor-mode :pin "6fa25a8e6b6a59481bc0354c2fe1e0ed53cbdc91")
(package! mips-mode :pin "98795cdc81979821ac35d9f94ce354cd99780c67")
(package! riscv-mode :pin "8e335b9c93de93ed8dd063d702b0f5ad48eef6d7")
(package! x86-lookup :pin "1573d61cc4457737b94624598a891c837fb52c16")

(use-package! nasm-mode
  :mode "\\.[n]*\\(asm\\|s\\)\\'")

;; Get Haxor VM from https://github.com/krzysztof-magosa/haxor
(use-package! haxor-mode
  :mode "\\.hax\\'")

(use-package! mips-mode
  :mode "\\.mips\\'")

(use-package! riscv-mode
  :mode "\\.riscv\\'")

(use-package! x86-lookup
  :commands (x86-lookup)
```

```
  :config
  (when (modulep! :tools pdf)
    (setq x86-lookup-browse-pdf-function 'x86-lookup-browse-pdf-pdf-tools))
  ;; Get manual from https://www.intel.com/content/www/us/en/developer/articles/techn
  (setq x86-lookup-pdf (expand-file-name "x86-lookup/325383-sdm-vol-2abcd.pdf" doom-d
```

10.8 Devdocs

```
(package! devdocs
  :recipe (:host github
           :repo "astoff/devdocs.el"
           :files ("*.el"))
  :pin "61ce83b79dc64e2f99d7f016a09b97e14b331459")

(use-package! devdocs
  :commands (devdocs-lookup devdocs-install)
  :config
  (setq devdocs-data-dir (expand-file-name "devdocs" doom-data-dir)))
```

10.9 PKGBUILD

```
(package! pkgbuild-mode :pin "9525be8ecbd3a0d0bc7cc27e6d0f403e111aa067")

(use-package! pkgbuild-mode
  :commands (pkgbuild-mode)
  :mode "/PKGBUILD$")
```

10.10 Flycheck + Projectile

WIP: Not working atm!

```
(package! flycheck-projectile
  :recipe (:host github
           :repo "nbfalcon/flycheck-projectile")
  :pin "ce6e9e8793a55dace13d5fa13badab2dca3b5ddb")

(use-package! flycheck-projectile
  :commands flycheck-projectile-list-errors)
```

10.11 Graphviz

Graphviz is a nice method of visualizing simple graphs, based on th DOT graph description language (`*.dot` / `*.gv` files).

```
(package! graphviz-dot-mode :pin "6e96a89762760935a7dff6b18393396f6498f976")
```

```
(use-package! graphviz-dot-mode
  :commands graphviz-dot-mode
  :mode ("\\.dot\\'" "\\.gv\\'")
  :init
  (after! org
    (setcdr (assoc "dot" org-src-lang-modes) 'graphviz-dot))

  :config
  (require 'company-graphviz-dot))
```

10.12 Mermaid

```
(package! mermaid-mode :pin "a98a9e733b1da1e6a19e68c1db4367bf46283479")

(package! ob-mermaid
  :recipe (:host github
           :repo "arnm/ob-mermaid")
  :pin "b4ce25699e3ebff054f523375d1cf5a17bd0dbaf")
```

```
(use-package! mermaid-mode
  :commands mermaid-mode
  :mode "\\.mmd\\'")

(use-package! ob-mermaid
  :after org
  :init
  (after! org
    (add-to-list 'org-babel-load-languages '(mermaid . t))))
```

10.13 LaTeX

```
(package! aas
  :recipe (:host github
           :repo "ymarco/auto-activating-snippets")
  :pin "566944e3b336c29d3ac11cd739a954c9d112f3fb")
```

```
(use-package! aas
  :commands aas-mode)
```

11 Office

11.1 Org additional packages

To avoid problems in the `(after! org)` section.

```
(unpin! org-roam) ;; To avoid problems with org-roam-ui
(package! websocket :pin "82b370602fa0158670b1c6c769f223159affce9b")
(package! org-roam-ui :pin "16a8da9e5107833032893bc4c0680b368ac423ac")
(package! org-wild-notifier :pin "9392b06d20b2f88e45a41bea17bb2f10f24fd19c")
(package! org-fragtog :pin "c675563af3f9ab5558cfd5ea460e2a07477b0cfd")
(package! org-appear :pin "60ba267c5da336e75e603f8c7ab3f44e6f4e4dac")
(package! org-super-agenda :pin "f4f528985397c833c870967884b013cf91a1da4a")
(package! doct :pin "506c22f365b75f5423810c4933856802554df464")

(package! citar-org-roam
  :recipe (:host github
           :repo "emacs-citar/citar-org-roam")
  :pin "29688b89ac3bf78405fa0dce7e17965aa8fe0dff")

(package! org-menu
  :recipe (:host github
           :repo "sheijk/org-menu")
  :pin "9cd10161c2b50dfef581f3d0441683eeeae6be59")

(package! caldav
  :recipe (:host github
           :repo "dengste/org-caldav")
  :pin "8569941a0a5a9393ba51afc8923fd7b77b73fa7a")

(package! org-ol-tree
  :recipe (:host github
           :repo "Townk/org-ol-tree")
  :pin "207c748aa5fea8626be619e8c55bdb1c16118c25")

(package! org-modern
  :recipe (:host github
```

```
          :repo "minad/org-modern")
  :pin "828cf100c62fc9dfb50152c192ac3a968c1b54bc")

(package! org-bib
  :recipe (:host github
           :repo "rougier/org-bib-mode")
  :pin "fed9910186e5e579c2391fb356f55ae24093b55a")

(package! academic-phrases
  :recipe (:host github
           :repo "nashamri/academic-phrases")
  :pin "25d9cf67feac6359cb213f061735e2679c84187f")

(package! phscroll
  :recipe (:host github
           :repo "misohena/phscroll")
  :pin "65e00c89f078997e1a5665d069ad8b1e3b851d49")
```

11.2 Org mode

11.2.1 Intro

Because this section is fairly expensive to initialize, we'll wrap it in a `(after! ...)` block.

```
(after! org
  <<org-conf>>
)
```

11.2.2 Behavior

Tweaking defaults

Org basics

```
(setq org-directory "~/Dropbox/Org/" ; let's put files here
      org-use-property-inheritance t ; it's convenient to have properties inherited
      org-log-done 'time             ; having the time an item is done sounds conveni
      org-list-allow-alphabetical t  ; have a. A. a) A) list bullets
      org-export-in-background nil   ; run export processes in external emacs process
      org-export-async-debug t
```

```
      org-tags-column 0
      org-catch-invisible-edits 'smart ;; try not to accidently do weird stuff in inv
      org-export-with-sub-superscripts '{} ;; don't treat lone _ / ^ as sub/superscri
      org-pretty-entities-include-sub-superscripts nil
      org-auto-align-tags nil
      org-special-ctrl-a/e t
      org-startup-indented t ;; Enable 'org-indent-mode' by default, override with '+
      org-insert-heading-respect-content t)
```

Babel I also like the `:comments` header-argument, so let's make that a default.

```
(setq org-babel-default-header-args
      '((:session  . "none")
        (:results  . "replace")
        (:exports  . "code")
        (:cache    . "no")
        (:noweb    . "no")
        (:hlines   . "no")
        (:tangle   . "no")
        (:comments . "link")))
```

Babel is really annoying when it comes to working with Scheme (via Geiser), it keeps asking about which Scheme implementation to use, I tried to set this as a local variable (using) and `.dir-locals.el`, but it didn't work. This hack should solve the problem now!

```
;; stolen from https://github.com/yohan-pereira/.emacs#babel-config
(defun +org-confirm-babel-evaluate (lang body)
  (not (string= lang "scheme"))) ;; Don't ask for scheme

(setq org-confirm-babel-evaluate #'+org-confirm-babel-evaluate)
```

EVIL There also seem to be a few keybindings which use `hjkl`, but miss arrow key equivalents.

```
(map! :map evil-org-mode-map
      :after evil-org
      :n "g <up>" #'org-backward-heading-same-level
      :n "g <down>" #'org-forward-heading-same-level
      :n "g <left>" #'org-up-element
      :n "g <right>" #'org-down-element)
```

TODOs

```
(setq org-todo-keywords
      '((sequence "IDEA(i)" "TODO(t)" "NEXT(n)" "PROJ(p)" "STRT(s)" "WAIT(w)" "HOLD(h
        (sequence "[ ](T)" "[-](S)" "|" "[X](D)")
        (sequence "|" "OKAY(o)" "YES(y)" "NO(n)")))

(setq org-todo-keyword-faces
      '(("IDEA" . (:foreground "goldenrod" :weight bold))
        ("NEXT" . (:foreground "IndianRed1" :weight bold))
        ("STRT" . (:foreground "OrangeRed" :weight bold))
        ("WAIT" . (:foreground "coral" :weight bold))
        ("KILL" . (:foreground "DarkGreen" :weight bold))
        ("PROJ" . (:foreground "LimeGreen" :weight bold))
        ("HOLD" . (:foreground "orange" :weight bold))))

(defun +log-todo-next-creation-date (&rest ignore)
  "Log NEXT creation time in the property drawer under the key 'ACTIVATED'"
  (when (and (string= (org-get-todo-state) "NEXT")
             (not (org-entry-get nil "ACTIVATED")))
    (org-entry-put nil "ACTIVATED" (format-time-string "[%Y-%m-%d]"))))

(add-hook 'org-after-todo-state-change-hook #'+log-todo-next-creation-date)
```

Tags

```
(setq org-tag-persistent-alist
      '((:startgroup . nil)
        ("home"      . ?h)
        ("research"  . ?r)
        ("work"      . ?w)
        (:endgroup   . nil)
        (:startgroup . nil)
        ("tool"      . ?o)
        ("dev"       . ?d)
        ("report"    . ?p)
        (:endgroup   . nil)
        (:startgroup . nil)
        ("easy"      . ?e)
        ("medium"    . ?m)
```

```
        ("hard"      . ?a)
        (:endgroup   . nil)
        ("urgent"    . ?u)
        ("key"       . ?k)
        ("bonus"     . ?b)
        ("ignore"    . ?i)
        ("noexport"  . ?x)))

(setq org-tag-faces
      '(("home"     . (:foreground "goldenrod"  :weight bold))
        ("research" . (:foreground "goldenrod"  :weight bold))
        ("work"     . (:foreground "goldenrod"  :weight bold))
        ("tool"     . (:foreground "IndianRed1" :weight bold))
        ("dev"      . (:foreground "IndianRed1" :weight bold))
        ("report"   . (:foreground "IndianRed1" :weight bold))
        ("urgent"   . (:foreground "red"        :weight bold))
        ("key"      . (:foreground "red"        :weight bold))
        ("easy"     . (:foreground "green4"     :weight bold))
        ("medium"   . (:foreground "orange"     :weight bold))
        ("hard"     . (:foreground "red"        :weight bold))
        ("bonus"    . (:foreground "goldenrod"  :weight bold))
        ("ignore"   . (:foreground "Gray"       :weight bold))
        ("noexport" . (:foreground "LimeGreen"  :weight bold))))
```

Agenda Set files for `org-agenda`

```
(setq org-agenda-files
      (list (expand-file-name "inbox.org" org-directory)
            (expand-file-name "agenda.org" org-directory)
            (expand-file-name "gcal-agenda.org" org-directory)
            (expand-file-name "notes.org" org-directory)
            (expand-file-name "projects.org" org-directory)
            (expand-file-name "archive.org" org-directory)))
```

Apply some styling on the standard agenda:

```
;; Agenda styling
(setq org-agenda-block-separator ?
      org-agenda-time-grid
```

```
      '((daily today require-timed)
        (800 1000 1200 1400 1600 1800 2000)
        "     " "        ")
      org-agenda-current-time-string
      "  now                            ")
```

Super agenda Configure `org-super-agenda`

```
(use-package! org-super-agenda
  :defer t
  :config
  (org-super-agenda-mode)
  :init
  (setq org-agenda-skip-scheduled-if-done t
        org-agenda-skip-deadline-if-done t
        org-agenda-include-deadlines t
        org-agenda-block-separator nil
        org-agenda-tags-column 100 ;; from testing this seems to be a good value
        org-agenda-compact-blocks t)

  (setq org-agenda-custom-commands
        '(("o" "Overview"
           ((agenda "" ((org-agenda-span 'day)
                        (org-super-agenda-groups
                         '((:name "Today"
                            :time-grid t
                            :date today
                            :todo "TODAY"
                            :scheduled today
                            :order 1)))))
            (alltodo "" ((org-agenda-overriding-header "")
                         (org-super-agenda-groups
                          '((:name "Next to do" :todo "NEXT" :order 1)
                            (:name "Important" :tag "Important" :priority "A" :order
                            (:name "Due Today" :deadline today :order 2)
                            (:name "Due Soon" :deadline future :order 8)
                            (:name "Overdue" :deadline past :face error :order 7)
                            (:name "Assignments" :tag "Assignment" :order 10)
                            (:name "Issues" :tag "Issue" :order 12)
                            (:name "Emacs" :tag "Emacs" :order 13)
```

```
(:name "Projects" :tag "Project" :order 14)
(:name "Research" :tag "Research" :order 15)
(:name "To read" :tag "Read" :order 30)
(:name "Waiting" :todo "WAIT" :order 20)
(:name "University" :tag "Univ" :order 32)
(:name "Trivial" :priority<= "E" :tag ("Trivial" "Unimpor
(:discard (:tag ("Chore" "Routine" "Daily")))))))))))
```

Calendar

Google calendar (`org-gcal`) I store my `org-gcal` configuration privately, it contains something like this:

```
(setq org-gcal-client-id "<SOME_ID>.apps.googleusercontent.com"
      org-gcal-client-secret "<SOME_SECRET>"
      org-gcal-fetch-file-alist '(("<USERNAME>@gmail.com" . "~/Dropbox/Org/gcal-agend

(after! org-gcal
  (load! "lisp/private/+org-gcal.el"))
```

TODO CalDAV Need to be configured, see the GitHub repo.

```
(use-package! caldav
  :commands (org-caldav-sync))
```

Capture Set capture files

```
(setq +org-capture-emails-file (expand-file-name "inbox.org" org-directory)
      +org-capture-todo-file (expand-file-name "inbox.org" org-directory)
      +org-capture-projects-file (expand-file-name "projects.org" org-directory))
```

Let's set up some `org-capture` templates, and make them visually nice to access.

```
(use-package! doct
  :commands (doct))

(after! org-capture
  <<prettify-capture>>
```

```
(defun +doct-icon-declaration-to-icon (declaration)
  "Convert :icon declaration to icon"
  (let ((name (pop declaration))
        (set  (intern (concat "all-the-icons-" (plist-get declaration :set))))
        (face (intern (concat "all-the-icons-" (plist-get declaration :color))))
        (v-adjust (or (plist-get declaration :v-adjust) 0.01)))
    (apply set `(,name :face ,face :v-adjust ,v-adjust))))

(defun +doct-iconify-capture-templates (groups)
  "Add declaration's :icon to each template group in GROUPS."
  (let ((templates (doct-flatten-lists-in groups)))
    (setq doct-templates
          (mapcar (lambda (template)
                    (when-let* ((props (nthcdr (if (= (length template) 4) 2 5) tem
                                (spec (plist-get (plist-get props :doct) :icon)))
                      (setf (nth 1 template) (concat (+doct-icon-declaration-to-ico
                                                     "\t"
                                                     (nth 1 template))))
                    template)
                  templates))))

(setq doct-after-conversion-functions '(+doct-iconify-capture-templates))

(defun set-org-capture-templates ()
  (setq org-capture-templates
        (doct `(("Personal todo" :keys "t"
                 :icon ("checklist" :set "octicon" :color "green")
                 :file +org-capture-todo-file
                 :prepend t
                 :headline "Inbox"
                 :type entry
                 :template ("* TODO %?"
                            "%i %a"))
                ("Personal note" :keys "n"
                 :icon ("sticky-note-o" :set "faicon" :color "green")
                 :file +org-capture-todo-file
                 :prepend t
                 :headline "Inbox"
                 :type entry
                 :template ("* %?"
```

```
            "%i %a"))
("Email" :keys "e"
 :icon ("envelope" :set "faicon" :color "blue")
 :file +org-capture-todo-file
 :prepend t
 :headline "Inbox"
 :type entry
 :template ("* TODO %^{type|reply to|contact} %\\3 %? :email:"
            "Send an email %^{urgancy|soon|ASAP|anon|at some point|
            "about %^{topic}"
            "%U %i %a"))
("Interesting" :keys "i"
 :icon ("eye" :set "faicon" :color "lcyan")
 :file +org-capture-todo-file
 :prepend t
 :headline "Interesting"
 :type entry
 :template ("* [ ] %{desc}%? :%{i-type}:"
            "%i %a")
 :children (("Webpage" :keys "w"
             :icon ("globe" :set "faicon" :color "green")
             :desc "%(org-cliplink-capture) "
             :i-type "read:web")
            ("Article" :keys "a"
             :icon ("file-text" :set "octicon" :color "yellow")
             :desc ""
             :i-type "read:reaserch")
            ("Information" :keys "i"
             :icon ("info-circle" :set "faicon" :color "blue")
             :desc ""
             :i-type "read:info")
            ("Idea" :keys "I"
             :icon ("bubble_chart" :set "material" :color "silver")
             :desc ""
             :i-type "idea")))
("Tasks" :keys "k"
 :icon ("inbox" :set "octicon" :color "yellow")
 :file +org-capture-todo-file
 :prepend t
 :headline "Tasks"
```

```
 :type entry
 :template ("* TODO %? %^G%{extra}"
            "%i %a")
 :children (("General Task" :keys "k"
             :icon ("inbox" :set "octicon" :color "yellow")
             :extra "")

            ("Task with deadline" :keys "d"
             :icon ("timer" :set "material" :color "orange" :v-adju
             :extra "\nDEADLINE: %^{Deadline:}t")

            ("Scheduled Task" :keys "s"
             :icon ("calendar" :set "octicon" :color "orange")
             :extra "\nSCHEDULED: %^{Start time:}t")))
("Project" :keys "p"
 :icon ("repo" :set "octicon" :color "silver")
 :prepend t
 :type entry
 :headline "Inbox"
 :template ("* %{time-or-todo} %?"
            "%i"
            "%a")
 :file ""
 :custom (:time-or-todo "")
 :children (("Project-local todo" :keys "t"
             :icon ("checklist" :set "octicon" :color "green")
             :time-or-todo "TODO"
             :file +org-capture-project-todo-file)
            ("Project-local note" :keys "n"
             :icon ("sticky-note" :set "faicon" :color "yellow")
             :time-or-todo "%U"
             :file +org-capture-project-notes-file)
            ("Project-local changelog" :keys "c"
             :icon ("list" :set "faicon" :color "blue")
             :time-or-todo "%U"
             :heading "Unreleased"
             :file +org-capture-project-changelog-file)))
("\tCentralised project templates"
 :keys "o"
 :type entry
```

```
                    :prepend t
                    :template ("* %{time-or-todo} %?"
                               "%i"
                               "%a")
                    :children (("Project todo"
                                :keys "t"
                                :prepend nil
                                :time-or-todo "TODO"
                                :heading "Tasks"
                                :file +org-capture-central-project-todo-file)
                               ("Project note"
                                :keys "n"
                                :time-or-todo "%U"
                                :heading "Notes"
                                :file +org-capture-central-project-notes-file)
                               ("Project changelog"
                                :keys "c"
                                :time-or-todo "%U"
                                :heading "Unreleased"
                                :file +org-capture-central-project-changelog-file)))))

  (set-org-capture-templates)
  (unless (display-graphic-p)
    (add-hook 'server-after-make-frame-hook
              (defun org-capture-reinitialise-hook ()
                (when (display-graphic-p)
                  (set-org-capture-templates)
                  (remove-hook 'server-after-make-frame-hook
                               #'org-capture-reinitialise-hook))))))
```

It would also be nice to improve how the capture dialogue looks

```
(defun org-capture-select-template-prettier (&optional keys)
  "Select a capture template, in a prettier way than default
Lisp programs can force the template by setting KEYS to a string."
  (let ((org-capture-templates
         (or (org-contextualize-keys
              (org-capture-upgrade-templates org-capture-templates)
              org-capture-templates-contexts)
             '(("t" "Task" entry (file+headline "" "Tasks")
```

```
                "* TODO %?\n  %u\n  %a")))))
    (if keys
        (or (assoc keys org-capture-templates)
            (error "No capture template referred to by \"%s\" keys" keys))
      (org-mks org-capture-templates
               "Select a capture template\n             "
               "Template key: "
               `(("q" ,(concat (all-the-icons-octicon "stop" :face 'all-the-icons-red
(advice-add 'org-capture-select-template :override #'org-capture-select-template-pret

(defun org-mks-pretty (table title &optional prompt specials)
  "Select a member of an alist with multiple keys. Prettified.

TABLE is the alist which should contain entries where the car is a string.
There should be two types of entries.

1. prefix descriptions like (\"a\" \"Description\")
   This indicates that `a' is a prefix key for multi-letter selection, and
   that there are entries following with keys like \"ab\", \"ax\"…

2. Select-able members must have more than two elements, with the first
   being the string of keys that lead to selecting it, and the second a
   short description string of the item.

The command will then make a temporary buffer listing all entries
that can be selected with a single key, and all the single key
prefixes.  When you press the key for a single-letter entry, it is selected.
When you press a prefix key, the commands (and maybe further prefixes)
under this key will be shown and offered for selection.

TITLE will be placed over the selection in the temporary buffer,
PROMPT will be used when prompting for a key.  SPECIALS is an
alist with (\"key\" \"description\") entries.  When one of these
is selected, only the bare key is returned."
  (save-window-excursion
    (let ((inhibit-quit t)
          (buffer (org-switch-to-buffer-other-window "*Org Select*"))
          (prompt (or prompt "Select: "))
          case-fold-search
          current)
```

```
(unwind-protect
    (catch 'exit
      (while t
        (setq-local evil-normal-state-cursor (list nil))
        (erase-buffer)
        (insert title "\n\n")
        (let ((des-keys nil)
              (allowed-keys '("\C-g"))
              (tab-alternatives '("\s" "\t" "\r"))
              (cursor-type nil))
          ;; Populate allowed keys and descriptions keys
          ;; available with CURRENT selector.
          (let ((re (format "\\`%s\\(.\\)\\'"
                            (if current (regexp-quote current) "")))
                (prefix (if current (concat current " ") "")))
            (dolist (entry table)
              (pcase entry
                ;; Description.
                (`(,(and key (pred (string-match re))) ,desc)
                 (let ((k (match-string 1 key)))
                   (push k des-keys)
                   ;; Keys ending in tab, space or RET are equivalent.
                   (if (member k tab-alternatives)
                       (push "\t" allowed-keys)
                     (push k allowed-keys))
                   (insert (propertize prefix 'face 'font-lock-comment-face) (p
                ;; Usable entry.
                (`(,(and key (pred (string-match re))) ,desc . ,_)
                 (let ((k (match-string 1 key)))
                   (insert (propertize prefix 'face 'font-lock-comment-face) (p
                   (push k allowed-keys)))
                (_ nil))))
          ;; Insert special entries, if any.
          (when specials
            (insert "             \n")
            (pcase-dolist (`(,key ,description) specials)
              (insert (format "%s   %s\n" (propertize key 'face '(bold all-the-
              (push key allowed-keys)))
          ;; Display UI and let user select an entry or
          ;; a sublevel prefix.
```

```
                (goto-char (point-min))
                (unless (pos-visible-in-window-p (point-max))
                  (org-fit-window-to-buffer))
                (let ((pressed (org--mks-read-key allowed-keys
                                                  prompt
                                                  (not (pos-visible-in-window-p (1- (
                  (setq current (concat current pressed))
                  (cond
                   ((equal pressed "\C-g") (user-error "Abort"))
                   ;; Selection is a prefix: open a new menu.
                   ((member pressed des-keys))
                   ;; Selection matches an association: return it.
                   ((let ((entry (assoc current table)))
                      (and entry (throw 'exit entry))))
                   ;; Selection matches a special entry: return the
                   ;; selection prefix.
                   ((assoc current specials) (throw 'exit current))
                   (t (error "No entry available")))))))
        (when buffer (kill-buffer buffer))))))
(advice-add 'org-mks :override #'org-mks-pretty)
```

The org-capture bin is rather nice, but I'd be nicer with a smaller frame, and no modeline.

```
(setf (alist-get 'height +org-capture-frame-parameters) 15)
;; (alist-get 'name +org-capture-frame-parameters) "  Capture") ;; ATM hardcoded in ot
(setq +org-capture-fn
      (lambda ()
        (interactive)
        (set-window-parameter nil 'mode-line-format 'none)
        (org-capture)))
```

Snippet Helpers I often want to set `src-block` headers, and it's a pain to:

- type them out
- remember what the accepted values are
- oh, and specifying the same language again and again

We can solve this in three steps:

- having one-letter snippets, conditioned on `(point)` being within a src header
- creating a nice prompt showing accepted values and the current default
- pre-filling the `src-block` language with the last language used

For header args, the keys I'll use are:

- `r` for `:results`
- `e` for `:exports`
- `v` for `:eval`
- `s` for `:session`
- `d` for `:dir`

```
(defun +yas/org-src-header-p ()
  "Determine whether `point' is within a src-block header or header-args."
  (pcase (org-element-type (org-element-context))
    ('src-block (< (point) ; before code part of the src-block
                   (save-excursion (goto-char (org-element-property :begin (org-eleme
                                   (forward-line 1)
                                   (point))))
    ('inline-src-block (< (point) ; before code part of the inline-src-block
                          (save-excursion (goto-char (org-element-property :begin (or
                                          (search-forward "]{")
                                          (point))))
    ('keyword (string-match-p "^header-args" (org-element-property :value (org-elemen
```

Now let's write a function we can reference in YASnippets to produce a nice interactive way to specify header arguments.

```
(defun +yas/org-prompt-header-arg (arg question values)
  "Prompt the user to set ARG header property to one of VALUES with QUESTION.
The default value is identified and indicated. If either default is selected,
or no selection is made: nil is returned."
  (let* ((src-block-p (not (looking-back "^#\\+property:[ \t]+header-args:.*" (line-b
         (default
           (or
            (cdr (assoc arg
```

```
                        (if src-block-p
                            (nth 2 (org-babel-get-src-block-info t))
                          (org-babel-merge-params
                           org-babel-default-header-args
                           (let ((lang-headers
                                  (intern (concat "org-babel-default-header-args:"
                                                  (+yas/org-src-lang)))))
                             (when (boundp lang-headers) (eval lang-headers t)))))))
            ""))
        default-value)
    (setq values (mapcar
                  (lambda (value)
                    (if (string-match-p (regexp-quote value) default)
                        (setq default-value
                              (concat value " "
                                      (propertize "(default)" 'face 'font-lock-doc-fa
                      value))
                  values))
    (let ((selection (consult--read question values :default default-value)))
      (unless (or (string-match-p "(default)$" selection)
                  (string= "" selection))
        selection))))
```

Finally, we fetch the language information for new source blocks.

Since we're getting this info, we might as well go a step further and also provide the ability to determine the most popular language in the buffer that doesn't have any `header-args` set for it (with `#+properties`).

```
(defun +yas/org-src-lang ()
  "Try to find the current language of the src/header at `point'.
Return nil otherwise."
  (let ((context (org-element-context)))
    (pcase (org-element-type context)
      ('src-block (org-element-property :language context))
      ('inline-src-block (org-element-property :language context))
      ('keyword (when (string-match "^header-args:\\([^ ]+\\)" (org-element-property
                  (match-string 1 (org-element-property :value context)))))))

(defun +yas/org-last-src-lang ()
  "Return the language of the last src-block, if it exists."
```

```
  (save-excursion
    (beginning-of-line)
    (when (re-search-backward "^[ \t]*#\\+begin_src" nil t)
      (org-element-property :language (org-element-context)))))

(defun +yas/org-most-common-no-property-lang ()
  "Find the lang with the most source blocks that has no global header-args, else nil
  (let (src-langs header-langs)
    (save-excursion
      (goto-char (point-min))
      (while (re-search-forward "^[ \t]*#\\+begin_src" nil t)
        (push (+yas/org-src-lang) src-langs))
      (goto-char (point-min))
      (while (re-search-forward "^[ \t]*#\\+property: +header-args" nil t)
        (push (+yas/org-src-lang) header-langs)))

    (setq src-langs
          (mapcar #'car
                  ;; sort alist by frequency (desc.)
                  (sort
                   ;; generate alist with form (value . frequency)
                   (cl-loop for (n . m) in (seq-group-by #'identity src-langs)
                            collect (cons n (length m)))
                   (lambda (a b) (> (cdr a) (cdr b))))))

    (car (cl-set-difference src-langs header-langs :test #'string=))))
```

Translate capital keywords to lower case Everyone used to use `#+CAPITAL` keywords. Then people realised that `#+lowercase` is actually both marginally easier and visually nicer, so now the capital version is just used in the manual.

> Org is standardized on lower case. Uppercase is used in the manual as a poor man's bold, and supported for historical reasons.
> — Nicolas Goaziou

```
(defun +org-syntax-convert-keyword-case-to-lower ()
  "Convert all #+KEYWORDS to #+keywords."
  (interactive)
  (save-excursion
```

```
    (goto-char (point-min))
    (let ((count 0)
          (case-fold-search nil))
      (while (re-search-forward "^[ \t]*#\\+[A-Z_]+" nil t)
        (unless (s-matches-p "RESULTS" (match-string 0))
          (replace-match (downcase (match-string 0)) t)
          (setq count (1+ count))))
      (message "Replaced %d occurances" count))))
```

Org notifier Add support for `org-wild-notifier`.

```
(use-package! org-wild-notifier
  :hook (org-load . org-wild-notifier-mode)
  :config
  (setq org-wild-notifier-alert-time '(60 30)))
```

Org menu

```
(use-package! org-menu
  :commands (org-menu)
  :init
  (map! :localleader
        :map org-mode-map
        :desc "Org menu" "M" #'org-menu))
```

LSP in src blocks

```
(when (and (modulep! :tools lsp) (not (modulep! :tools lsp +eglot)))
  (cl-defmacro +lsp-org-babel-enable (lang)
    "Support LANG in org source code block."
    ;; (setq centaur-lsp 'lsp-mode)
    (cl-check-type lang stringp)
    (let* ((edit-pre (intern (format "org-babel-edit-prep:%s" lang)))
           (intern-pre (intern (format "lsp--%s" (symbol-name edit-pre)))))
      `(progn
         (defun ,intern-pre (info)
           (let ((file-name (->> info caddr (alist-get :file))))
             (unless file-name
               (setq file-name (make-temp-file "babel-lsp-")))
             (setq buffer-file-name file-name)
```

```
            (lsp-deferred)))
        (put ',intern-pre 'function-documentation
             (format "Enable lsp-mode in the buffer of org source block (%s)."
                     (upcase ,lang)))
        (if (fboundp ',edit-pre)
            (advice-add ',edit-pre :after ',intern-pre)
          (progn
            (defun ,edit-pre (info)
              (,intern-pre info))
            (put ',edit-pre 'function-documentation
                 (format "Prepare local buffer environment for org source block (%s)
                         (upcase ,lang))))))))

  (defvar +org-babel-lang-list
    '("go" "python" "ipython" "bash" "sh"))

  (dolist (lang +org-babel-lang-list)
    (eval `(+lsp-org-babel-enable ,lang))))
```

11.2.3 Custom links

Sub-figures This defines a new link type `subfig` to enable exporting sub-figures to LaTeX, taken form "Export subfigures to LaTeX (and HTML)".

```
(org-link-set-parameters
 "subfig"
 :follow (lambda (file) (find-file file))
 :face '(:foreground "chocolate" :weight bold :underline t)
 :display 'full
 :export
 (lambda (file desc backend)
   (when (eq backend 'latex)
     (if (string-match ">(\\(.+\\))" desc)
         (concat "\\begin{subfigure}[b]"
                 "\\caption{" (replace-regexp-in-string "\s+>(.+)" "" desc) "}"
                 "\\includegraphics" "[" (match-string 1 desc) "]" "{" file "}" "\\en
       (format "\\begin{subfigure}\\includegraphics{%s}\\end{subfigure}" desc file)))
```

Example of usage:

```
#+caption: Lorem impsum dolor
```

```
#+attr_latex: :options \centering
#+begin_figure
[[subfig:img1.jpg][Caption of img1 >(width=.3\textwidth)]]

[[subfig:img2.jpg][Caption of img2 >(width=.3\textwidth)]]

[[subfig:img3.jpg][Caption of img3 >(width=.6\textwidth)]]
#+end_figure
```

LaTeX inline markup Needs to make a ?, with this hack you can write `[[latex:textsc][Some text]]`.

```
(org-add-link-type
 "latex" nil
 (lambda (path desc format)
   (cond
    ((eq format 'html)
     (format "<span class=\"%s\">%s</span>" path desc))
    ((eq format 'latex)
     (format "\\%s{%s}" path desc)))))
```

11.2.4 Visuals

Here I try to do two things: improve the styling of the various documents, via font changes etc., and also propagate colours from the current theme.

Font display

Headings Let's make the title and the headings a bit bigger:

```
(custom-set-faces!
  '(org-document-title :height 1.2))

(custom-set-faces!
  '(outline-1 :weight extra-bold :height 1.25)
  '(outline-2 :weight bold :height 1.15)
  '(outline-3 :weight bold :height 1.12)
  '(outline-4 :weight semi-bold :height 1.09)
  '(outline-5 :weight semi-bold :height 1.06)
  '(outline-6 :weight semi-bold :height 1.03)
```

```
  '(outline-8 :weight semi-bold)
  '(outline-9 :weight semi-bold))
```

Deadlines It seems reasonable to have deadlines in the error face when they're passed.

```
(setq org-agenda-deadline-faces
      '((1.001 . error)
        (1.000 . org-warning)
        (0.500 . org-upcoming-deadline)
        (0.000 . org-upcoming-distant-deadline)))
```

Font styling We can then have quote blocks stand out a bit more by making them *italic*.

```
(setq org-fontify-quote-and-verse-blocks t)
```

While `org-hide-emphasis-markers` is very nice, it can sometimes make edits which occur at the border a bit more fiddley. We can improve this situation without sacrificing visual amenities with the `org-appear` package.

```
(use-package! org-appear
  :hook (org-mode . org-appear-mode)
  :config
  (setq org-appear-autoemphasis t
        org-appear-autosubmarkers t
        org-appear-autolinks nil)
  ;; for proper first-time setup, `org-appear--set-elements'
  ;; needs to be run after other hooks have acted.
  (run-at-time nil nil #'org-appear--set-elements))
```

Inline blocks

```
(setq org-inline-src-prettify-results '(" " . " ")
      doom-themes-org-fontify-special-tags nil)
```

Image previews

```
(defvar +org-responsive-image-percentage 0.4)
(defvar +org-responsive-image-width-limits '(400 . 700)) ;; '(min-width . max-width)
```

```
(defun +org--responsive-image-h ()
  (when (eq major-mode 'org-mode)
    (setq org-image-actual-width
          (max (car +org-responsive-image-width-limits)
               (min (cdr +org-responsive-image-width-limits)
                    (truncate (* (window-pixel-width) +org-responsive-image-percentag

(add-hook 'window-configuration-change-hook #'+org--responsive-image-h)
```

Org Modern

```
(use-package! org-modern
  :hook (org-mode . org-modern-mode)
  :config
  (setq org-modern-star '(" " " " " " " " " " " " " ")
        org-modern-table-vertical 5
        org-modern-table-horizontal 2
        org-modern-list '((43 . " ") (45 . "-") (42 . "•"))
        org-modern-footnote (cons nil (cadr org-script-display))
        org-modern-priority t
        org-modern-block t
        org-modern-block-fringe nil
        org-modern-horizontal-rule t
        org-modern-keyword
        '((t                      . t)
          ("title"                . " ")
          ("subtitle"             . " ")
          ("author"               . " ")
          ("email"                . "@")
          ("date"                 . " ")
          ("lastmod"              . " ")
          ("property"             . " ")
          ("options"              . " ")
          ("startup"              . " ")
          ("macro"                . " ")
          ("bind"                 . #(" " 0 1 (display (raise -0.1))))
          ("bibliography"         . " ")
          ("print_bibliography"   . #(" " 0 1 (display (raise -0.1))))
          ("cite_export"          . " ")
          ("print_glossary"       . #("  " 0 1 (display (raise -0.1))))
```

```
          ("glossary_sources"    . #(" " 0 1 (display (raise -0.14))))
          ("export_file_name"    . " ")
          ("include"             . " ")
          ("setupfile"           . " ")
          ("html_head"           . " ")
          ("html"                . " ")
          ("latex_class"         . " ")
          ("latex_class_options" . #(" " 1 2 (display (raise -0.14))))
          ("latex_header"        . " ")
          ("latex_header_extra"  . " ")
          ("latex"               . " ")
          ("beamer_theme"        . " ")
          ("beamer_color_theme"  . #(" " 1 2 (display (raise -0.12))))
          ("beamer_font_theme"   . " ")
          ("beamer_header"       . " ")
          ("beamer"              . " ")
          ("attr_latex"          . " ")
          ("attr_html"           . " ")
          ("attr_org"            . " ")
          ("name"                . " ")
          ("header"              . ">")
          ("caption"             . " ")
          ("RESULTS"             . " ")
          ("language"            . " ")
          ("hugo_base_dir"       . " ")
          ("latex_compiler"      . " ")
          ("results"             . " ")
          ("filetags"            . "#")
          ("created"             . " ")
          ("export_select_tags"  . " ")
          ("export_exclude_tags" . " ")))

  ;; Change faces
  (custom-set-faces! '(org-modern-tag :inherit (region org-modern-label)))
  (custom-set-faces! '(org-modern-statistics :inherit org-checkbox-statistics-todo)))
```

We'll bind this to `O` on the `org-mode` localleader, and manually apply a PR recognising the pgtk window system.

```
(use-package! org-ol-tree
```

```
  :commands org-ol-tree
  :config
  (setq org-ol-tree-ui-icon-set
        (if (and (display-graphic-p)
                 (fboundp 'all-the-icons-material))
            'all-the-icons
          'unicode))
  (org-ol-tree-ui--update-icon-set))

(map! :localleader
      :map org-mode-map
      :desc "Outline" "O" #'org-ol-tree)
```

List bullet sequence I think it makes sense to have list bullets change with depth

```
(setq org-list-demote-modify-bullet
      '(("+"  . "-")
        ("-"  . "+")
        ("*"  . "+")
        ("1." . "a.")))
```

Symbols

```
;; Org styling, hide markup etc.
(setq org-hide-emphasis-markers t
      org-pretty-entities t
      org-ellipsis "  "
      org-hide-leading-stars t)
      ;; org-priority-highest ?A
      ;; org-priority-lowest ?E
      ;; org-priority-faces
      ;; '((?A . 'all-the-icons-red)
      ;;   (?B . 'all-the-icons-orange)
      ;;   (?C . 'all-the-icons-yellow)
      ;;   (?D . 'all-the-icons-green)
      ;;   (?E . 'all-the-icons-blue)))
```

LaTeX fragments

Prettier highlighting First off, we want those fragments to look good.

```
(setq org-highlight-latex-and-related '(native script entities))

(require 'org-src)
(add-to-list 'org-src-block-faces '("latex" (:inherit default :extend t)))
```

Prettier rendering Since we can, instead of making the background color match the `default` face, let's make it transparent.

```
(setq org-format-latex-options
      (plist-put org-format-latex-options :background "Transparent"))

;; Can be dvipng, dvisvgm, imagemagick
(setq org-preview-latex-default-process 'dvisvgm)

;; Define a function to set the format latex scale (to be reused in hooks)
(defun +org-format-latex-set-scale (scale)
  (setq org-format-latex-options (plist-put org-format-latex-options :scale scale)))

;; Set the default scale
(+org-format-latex-set-scale 1.4)
```

Better equation numbering Numbered equations all have (1) as the number for fragments with vanilla `org-mode`. This code (from scimax) injects the correct numbers into the previews, so they look good.

> This hack is not properly working right now!, it seems to work only with `align` blocks. **NEEDS INVESTIGATION.**

```
(defun +parse-the-fun (str)
  "Parse the LaTeX environment STR.
Return an AST with newlines counts in each level."
  (let (ast)
    (with-temp-buffer
      (insert str)
      (goto-char (point-min))
      (while (re-search-forward
              (rx "\\"
```

```
                    (group (or "\\" "begin" "end" "nonumber"))
                    (zero-or-one "{" (group (zero-or-more not-newline)) "}"))
                nil t)
          (let ((cmd (match-string 1))
                (env (match-string 2)))
            (cond ((string= cmd "begin")
                   (push (list :env (intern env)) ast))
                  ((string= cmd "\\")
                   (let ((curr (pop ast)))
                     (push (plist-put curr :newline (1+ (or (plist-get curr :newline) 0
                  ((string= cmd "nonumber")
                   (let ((curr (pop ast)))
                     (push (plist-put curr :nonumber (1+ (or (plist-get curr :nonumber)
                  ((string= cmd "end")
                   (let ((child (pop ast))
                         (parent (pop ast)))
                     (push (plist-put parent :childs (cons child (plist-get parent :chi
    (plist-get (car ast) :childs)))

(defun +scimax-org-renumber-environment (orig-func &rest args)
  "A function to inject numbers in LaTeX fragment previews."
  (let ((results '())
        (counter -1))
    (setq results
          (cl-loop for (begin . env) in
                   (org-element-map (org-element-parse-buffer) 'latex-environment
                     (lambda (env)
                       (cons
                        (org-element-property :begin env)
                        (org-element-property :value env))))
                   collect
                   (cond
                    ((and (string-match "\\\\begin{equation}" env)
                          (not (string-match "\\\\tag{" env)))
                     (cl-incf counter)
                     (cons begin counter))
                    ((string-match "\\\\begin{align}" env)
                     (cl-incf counter)
                     (let ((p (car (+parse-the-fun env))))
                       ;; Parse the `env', count new lines in the align env as equati
```

```
                      (cl-incf counter (- (or (plist-get p :newline) 0)
                                          (or (plist-get p :nonumber) 0))))
                    (cons begin counter))
                   (t
                    (cons begin nil)))))
    (when-let ((number (cdr (assoc (point) results))))
      (setf (car args)
            (concat
             (format "\\setcounter{equation}{%s}\n" number)
             (car args)))))
  (apply orig-func args))

(defun +scimax-toggle-latex-equation-numbering (&optional enable)
  "Toggle whether LaTeX fragments are numbered."
  (interactive)
  (if (or enable (not (get '+scimax-org-renumber-environment 'enabled)))
      (progn
        (advice-add 'org-create-formula-image :around #'+scimax-org-renumber-environm
        (put '+scimax-org-renumber-environment 'enabled t)
        (message "LaTeX numbering enabled."))
    (advice-remove 'org-create-formula-image #'+scimax-org-renumber-environment)
    (put '+scimax-org-renumber-environment 'enabled nil)
    (message "LaTeX numbering disabled.")))

(defun +scimax-org-inject-latex-fragment (orig-func &rest args)
  "Advice function to inject latex code before and/or after the equation in a latex f
You can use this to set \\mathversion{bold} for example to make
it bolder. The way it works is by defining
:latex-fragment-pre-body and/or :latex-fragment-post-body in the
variable `org-format-latex-options'. These strings will then be
injected before and after the code for the fragment before it is
made into an image."
  (setf (car args)
        (concat
         (or (plist-get org-format-latex-options :latex-fragment-pre-body) "")
         (car args)
         (or (plist-get org-format-latex-options :latex-fragment-post-body) "")))
  (apply orig-func args))

(defun +scimax-toggle-inject-latex ()
```

```
  "Toggle whether you can insert latex in fragments."
  (interactive)
  (if (not (get '+scimax-org-inject-latex-fragment 'enabled))
      (progn
        (advice-add 'org-create-formula-image :around #'+scimax-org-inject-latex-frag
        (put '+scimax-org-inject-latex-fragment 'enabled t)
        (message "Inject latex enabled"))
    (advice-remove 'org-create-formula-image #'+scimax-org-inject-latex-fragment)
    (put '+scimax-org-inject-latex-fragment 'enabled nil)
    (message "Inject latex disabled")))

;; Enable renumbering by default
(+scimax-toggle-latex-equation-numbering t)
```

Fragtog Hook `org-fragtog-mode` to `org-mode`.

```
(use-package! org-fragtog
  :hook (org-mode . org-fragtog-mode))
```

Org plot We can use some variables in `org-plot` to use the current doom theme colors.

```
(after! org-plot
  (defun org-plot/generate-theme (_type)
    "Use the current Doom theme colours to generate a GnuPlot preamble."
    (format "
fgt = \"textcolor rgb '%s'\"  # foreground text
fgat = \"textcolor rgb '%s'\" # foreground alt text
fgl = \"linecolor rgb '%s'\"  # foreground line
fgal = \"linecolor rgb '%s'\" # foreground alt line

# foreground colors
set border lc rgb '%s'
# change text colors of  tics
set xtics @fgt
set ytics @fgt
# change text colors of labels
set title @fgt
set xlabel @fgt
set ylabel @fgt
```

```
# change a text color of key
set key @fgt

# line styles
set linetype 1 lw 2 lc rgb '%s' # red
set linetype 2 lw 2 lc rgb '%s' # blue
set linetype 3 lw 2 lc rgb '%s' # green
set linetype 4 lw 2 lc rgb '%s' # magenta
set linetype 5 lw 2 lc rgb '%s' # orange
set linetype 6 lw 2 lc rgb '%s' # yellow
set linetype 7 lw 2 lc rgb '%s' # teal
set linetype 8 lw 2 lc rgb '%s' # violet

# palette
set palette maxcolors 8
set palette defined ( 0 '%s',\
1 '%s',\
2 '%s',\
3 '%s',\
4 '%s',\
5 '%s',\
6 '%s',\
7 '%s' )
"
            (doom-color 'fg)
            (doom-color 'fg-alt)
            (doom-color 'fg)
            (doom-color 'fg-alt)
            (doom-color 'fg)
            ;; colours
            (doom-color 'red)
            (doom-color 'blue)
            (doom-color 'green)
            (doom-color 'magenta)
            (doom-color 'orange)
            (doom-color 'yellow)
            (doom-color 'teal)
            (doom-color 'violet)
            ;; duplicated
            (doom-color 'red)
```

```
            (doom-color 'blue)
            (doom-color 'green)
            (doom-color 'magenta)
            (doom-color 'orange)
            (doom-color 'yellow)
            (doom-color 'teal)
            (doom-color 'violet)))

  (defun org-plot/gnuplot-term-properties (_type)
    (format "background rgb '%s' size 1050,650"
            (doom-color 'bg)))

  (setq org-plot/gnuplot-script-preamble #'org-plot/generate-theme
        org-plot/gnuplot-term-extra #'org-plot/gnuplot-term-properties))
```

Large tables Use *Partial Horizontal Scroll* to display long tables without breaking them.

```
(use-package! org-phscroll
  :hook (org-mode . org-phscroll-mode))
```

11.2.5 Bibliography

BibTeX

```
(setq bibtex-completion-bibliography +my/biblio-libraries-list
      bibtex-completion-library-path +my/biblio-storage-list
      bibtex-completion-notes-path +my/biblio-notes-path
      bibtex-completion-notes-template-multiple-files "* ${author-or-editor}, ${title
      bibtex-completion-additional-search-fields '(keywords)
      bibtex-completion-display-formats
      '((article       . "${=has-pdf=:1}${=has-note=:1} ${year:4} ${author:36} ${titl
        (inbook        . "${=has-pdf=:1}${=has-note=:1} ${year:4} ${author:36} ${titl
        (incollection  . "${=has-pdf=:1}${=has-note=:1} ${year:4} ${author:36} ${titl
        (inproceedings . "${=has-pdf=:1}${=has-note=:1} ${year:4} ${author:36} ${titl
        (t             . "${=has-pdf=:1}${=has-note=:1} ${year:4} ${author:36} ${titl
      bibtex-completion-pdf-open-function
      (lambda (fpath)
        (call-process "open" nil 0 nil fpath)))
```

Org-bib A mode to work with annotated bibliography in Org-Mode. See the repo for an example.

```
(use-package! org-bib
  :commands (org-bib-mode))
```

Org-cite

```
(after! oc
  (setq org-cite-csl-styles-dir +my/biblio-styles-path)
        ;; org-cite-global-bibliography +my/biblio-libraries-list)

  (defun +org-ref-to-org-cite ()
    "Simple conversion of org-ref citations to org-cite syntax."
    (interactive)
    (save-excursion
      (goto-char (point-min))
      (while (re-search-forward "\\[cite\\(.*\\):\\([^]]*\\)\\]" nil t)
        (let* ((old (substring (match-string 0) 1 (1- (length (match-string 0)))))
               (new (s-replace "&" "@" old)))
          (message "Replaced citation %s with %s" old new)
          (replace-match new))))))
```

Citar

```
(after! citar
  (setq citar-library-paths +my/biblio-storage-list
        citar-notes-paths  (list +my/biblio-notes-path)
        citar-bibliography  +my/biblio-libraries-list
        citar-symbol-separator "  ")

  (when (display-graphic-p)
    (setq citar-symbols
          `((file ,(all-the-icons-octicon "file-pdf"      :face 'error) . " ")
            (note ,(all-the-icons-octicon "file-text"     :face 'warning) . " ")
            (link ,(all-the-icons-octicon "link-external" :face 'org-link) . " "))))))

(use-package! citar-org-roam
  :after citar org-roam
  :no-require
```

```
  :config (citar-org-roam-mode)
  :init
  ;; Modified form: https://jethrokuan.github.io/org-roam-guide/
  (defun +org-roam-node-from-cite (entry-key)
    (interactive (list (citar-select-ref)))
    (let ((title (citar-format--entry
                  "${author editor} (${date urldate}) :: ${title}"
                  (citar-get-entry entry-key))))
      (org-roam-capture- :templates
                         '(("r" "reference" plain
                            "%?"
                            :if-new (file+head "references/${citekey}.org"
                                               ":properties:
:roam_refs: [cite:@${citekey}]
:end:
#+title: ${title}\n")
                            :immediate-finish t
                            :unnarrowed t))
                         :info (list :citekey entry-key)
                         :node (org-roam-node-create :title title)
                         :props '(:finalize find-file)))))
```

11.2.6 Exporting

General settings By default, Org only exports the first three levels of headings as *headings*, the rest is considered as paragraphs. Let's increase this to 5 levels.

```
(setq org-export-headline-levels 5)
```

Let's make use of the `:ignore:` tag from `ox-extra`, which provides a way to ignore exporting a heading, while exporting the content residing under it (different from `:noexport:`).

```
(require 'ox-extra)
(ox-extras-activate '(ignore-headlines))

(setq org-export-creator-string
      (format "Made with Emacs %s and Org %s" emacs-version (org-release)))
```

LaTeX export

Compiling

```
;; `org-latex-compilers' contains a list of possible values for the `%latex' argument
(setq org-latex-pdf-process
      '("latexmk -shell-escape -pdf -quiet -f -%latex -interaction=nonstopmode -outpu
```

Org LaTeX packages

```
;; 'svg' package depends on inkscape, imagemagik and ghostscript
(when (+all (mapcar 'executable-find '("inkscape" "magick" "gs")))
  (add-to-list 'org-latex-packages-alist '("" "svg")))

(add-to-list 'org-latex-packages-alist '("svgnames" "xcolor"))
;; (add-to-list 'org-latex-packages-alist '("" "fontspec")) ;; for xelatex
;; (add-to-list 'org-latex-packages-alist '("utf8" "inputenc"))
```

Export PDFs with syntax highlighting This is for code syntax highlighting in export. You need to use `-shell-escape` with latex, and install the `python-pygments` package.

```
;; Should be configured per document, as a local variable
;; (setq org-latex-listings 'minted)
;; (add-to-list 'org-latex-packages-alist '("" "minted"))

;; Default `minted` options, can be overwritten in file/dir locals
(setq org-latex-minted-options
      '(("frame"         "lines")
        ("fontsize"      "\\footnotesize")
        ("tabsize"       "2")
        ("breaklines"    "true")
        ("breakanywhere" "true") ;; break anywhere, no just on spaces
        ("style"         "default")
        ("bgcolor"       "GhostWhite")
        ("linenos"       "true")))

;; Link some org-mode blocks languages to lexers supported by minted
;; via (pygmentize), you can see supported lexers by running this command
;; in a terminal: `pygmentize -L lexers'
(dolist (pair '((ipython    "python")
                (jupyter    "python")
```

```
                (scheme     "scheme")
                (lisp-data  "lisp")
                (conf-unix  "unixconfig")
                (conf-space "unixconfig")
                (authinfo   "unixconfig")
                (gdb-script "unixconfig")
                (conf-toml  "yaml")
                (conf       "ini")
                (gitconfig  "ini")
                (systemd    "ini")))
  (unless (member pair org-latex-minted-langs)
    (add-to-list 'org-latex-minted-langs pair)))
```

Class templates

```
(after! ox-latex
  (add-to-list
   'org-latex-classes
   '("scr-article"
     "\\documentclass{scrartcl}"
     ("\\section{%s}"       . "\\section*{%s}")
     ("\\subsection{%s}"    . "\\subsection*{%s}")
     ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
     ("\\paragraph{%s}"     . "\\paragraph*{%s}")
     ("\\subparagraph{%s}"  . "\\subparagraph*{%s}")))

  (add-to-list
   'org-latex-classes
   '("lettre"
     "\\documentclass{lettre}"
     ("\\section{%s}"       . "\\section*{%s}")
     ("\\subsection{%s}"    . "\\subsection*{%s}")
     ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
     ("\\paragraph{%s}"     . "\\paragraph*{%s}")
     ("\\subparagraph{%s}"  . "\\subparagraph*{%s}")))

  (add-to-list
   'org-latex-classes
   '("blank"
     "[NO-DEFAULT-PACKAGES]\n[NO-PACKAGES]\n[EXTRA]"
```

```
   ("\\section{%s}"       . "\\section*{%s}")
   ("\\subsection{%s}"    . "\\subsection*{%s}")
   ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
   ("\\paragraph{%s}"     . "\\paragraph*{%s}")
   ("\\subparagraph{%s}"  . "\\subparagraph*{%s}")))

(add-to-list
 'org-latex-classes
 '("IEEEtran"
   "\\documentclass{IEEEtran}"
   ("\\section{%s}"       . "\\section*{%s}")
   ("\\subsection{%s}"    . "\\subsection*{%s}")
   ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
   ("\\paragraph{%s}"     . "\\paragraph*{%s}")
   ("\\subparagraph{%s}"  . "\\subparagraph*{%s}")))

(add-to-list
 'org-latex-classes
 '("ieeeconf"
   "\\documentclass{ieeeconf}"
   ("\\section{%s}"       . "\\section*{%s}")
   ("\\subsection{%s}"    . "\\subsection*{%s}")
   ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
   ("\\paragraph{%s}"     . "\\paragraph*{%s}")
   ("\\subparagraph{%s}"  . "\\subparagraph*{%s}")))

(add-to-list
 'org-latex-classes
 '("sagej"
   "\\documentclass{sagej}"
   ("\\section{%s}"       . "\\section*{%s}")
   ("\\subsection{%s}"    . "\\subsection*{%s}")
   ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
   ("\\paragraph{%s}"     . "\\paragraph*{%s}")
   ("\\subparagraph{%s}"  . "\\subparagraph*{%s}")))

(add-to-list
 'org-latex-classes
 '("thesis"
   "\\documentclass[11pt]{book}"
```

```
    ("\\chapter{%s}"       . "\\chapter*{%s}")
    ("\\section{%s}"       . "\\section*{%s}")
    ("\\subsection{%s}"    . "\\subsection*{%s}")
    ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
    ("\\paragraph{%s}"     . "\\paragraph*{%s}")))

  (add-to-list
   'org-latex-classes
   '("thesis-fr"
     "\\documentclass[french,12pt,a4paper]{book}"
     ("\\chapter{%s}"       . "\\chapter*{%s}")
     ("\\section{%s}"       . "\\section*{%s}")
     ("\\subsection{%s}"    . "\\subsection*{%s}")
     ("\\subsubsection{%s}" . "\\subsubsection*{%s}")
     ("\\paragraph{%s}"     . "\\paragraph*{%s}"))))

(setq org-latex-default-class "article")

;; org-latex-tables-booktabs t
;; org-latex-reference-command "\\cref{%s}")
```

Export multi-files Org documents Let's say we have a multi-files document, with `main.org` as the entry point. Supposing a document with a structure like this:

[width=0.55]assets/multi-files-org

Figure 1: Example of a multi-files document structure

Files `intro.org`, `chap1.org`, … are included in `main.org` using the Org command . In such a setup, we will spend most of our time writing in a chapter files, and not the `main.org`, where when want to export the document, we would need to open the top-level file `main.org` before exporting.

A quick solution is **to admit the following convention**:

> If a file named `main.org` is present beside any other Org file, it should be considered as the entry point; and whenever we export to PDF (from any of the Org files like: `intro.org`, `chap1.org`, …), we automatically jump to the `main.org`, and run the export there.

This can be achieved by adding an Emacs-Lisp *advice* around the `(org-latex-export-to-pdf)` to switch to `main.org` (if it exists) before running the export.

You can also set the variable `+org-export-to-pdf-main-file` to the main file, in `.dir-locals.el` or as a file local variable.

```
(defvar +org-export-to-pdf-main-file nil
  "The main (entry point) Org file for a multi-files document.")

(advice-add
 'org-latex-export-to-pdf :around
 (lambda (orig-fn &rest orig-args)
   (message
    "PDF exported to: %s."
    (let ((main-file (or (bound-and-true-p +org-export-to-pdf-main-file) "main.org"))
      (if (file-exists-p (expand-file-name main-file))
          (with-current-buffer (find-file-noselect main-file)
            (apply orig-fn orig-args))
        (apply orig-fn orig-args))))))
```

Hugo Update files with last modified date, when `#+lastmod:` is available

```
(setq time-stamp-active t
      time-stamp-start  "#\\+lastmod:[ \t]*"
      time-stamp-end    "$"
      time-stamp-format "%04Y-%02m-%02d")

(add-hook 'before-save-hook 'time-stamp nil)
(setq org-hugo-auto-set-lastmod t)
```

11.3 Text editing

11.3.1 Plain text

It's nice to see ANSI color codes displayed. However, until Emacs 28 it's not possible to do this without modifying the buffer, so let's condition this block on that.

```
(after! text-mode
  (add-hook! 'text-mode-hook
    (unless (derived-mode-p 'org-mode)
      ;; Apply ANSI color codes
```

```
      (with-silent-modifications
        (ansi-color-apply-on-region (point-min) (point-max) t)))))
```

11.3.2 Academic phrases

When writing your academic paper, you might get stuck trying to find the right phrase that captures your intention. This package tries to alleviate that problem by presenting you with a list of phrases organized by the topic or by the paper section that you are writing. This package has around 600 phrases so far.

This is based on the book titled "English for Writing Research - Papers Useful Phrases".

```
(use-package! academic-phrases
  :commands (academic-phrases
             academic-phrases-by-section))
```

11.3.3 Yanking multi-lines paragraphs

```
(defun +helper-paragraphized-yank ()
  "Copy, then remove newlines and Org styling (/*_~)."
  (interactive)
  (copy-region-as-kill nil nil t)
  (with-temp-buffer
    (yank)
    ;; Remove newlines, and Org styling (/*_~)
    (goto-char (point-min))
    (let ((case-fold-search nil))
      (while (re-search-forward "[\n/*_~]" nil t)
        (replace-match (if (s-matches-p (match-string 0) "\n") " " "") t)))
    (kill-region (point-min) (point-max))))

(map! :localleader
      :map (org-mode-map markdown-mode-map latex-mode-map text-mode-map)
      :desc "Paragraphized yank" "y" #'+helper-paragraphized-yank)
```

12 Other Config

12.1 Rust format

For Rust code base, the file `$HOME/.rustfmt.toml` contains the global format settings, I like to set it to:

```
# Rust edition 2021
edition = "2021"

# Use Unix style newlines, with 2 spaces tabulation.
hard_tabs = false

# Make one line functions in a single line
fn_single_line = true

# Format strings
format_strings = true

# Increase the max line width
max_width = 120

# Merge nested imports
merge_imports = true

# Enum and Struct alignement
enum_discrim_align_threshold = 20
struct_field_align_threshold = 20

# Reorder impl items: type > const > macros > methods.
reorder_impl_items = true

# Comments and documentation formating
wrap_comments = true
normalize_comments = true
normalize_doc_attributes = true
format_code_in_doc_comments = true
report_fixme = "Always"
todo = "Always"
```

12.2 Enabled some packages

```
(package! editorconfig)
(package! markdown-preview-mode)
```